MACHINE LEARNING

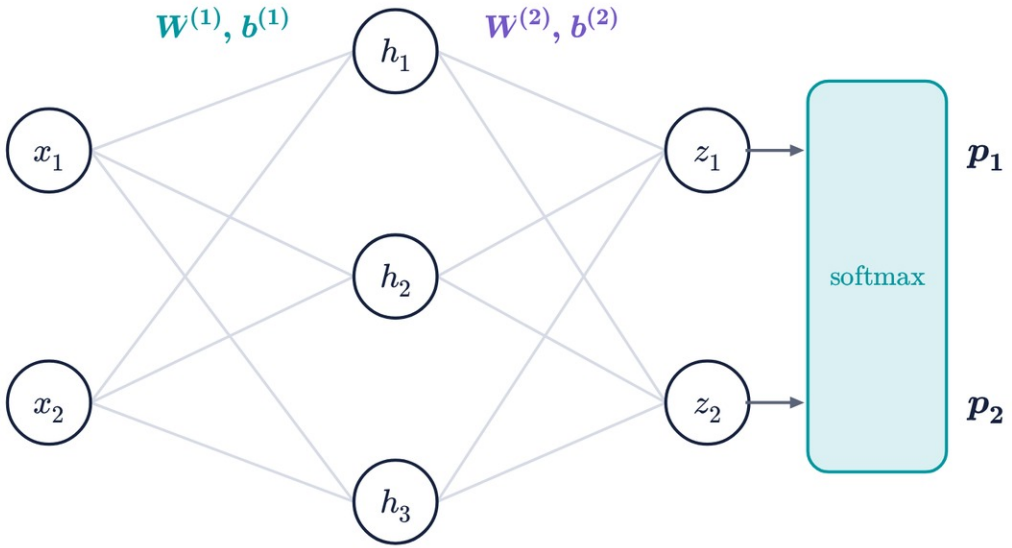
Neural networks

Part 2: training a network.

LAST TIME

Forward propagation

$$h = \sigma(W^{(1)}x + b^{(1)})$$
$$z = W^{(2)}h + b^{(2)}$$
$$p = \text{softmax}(z)$$



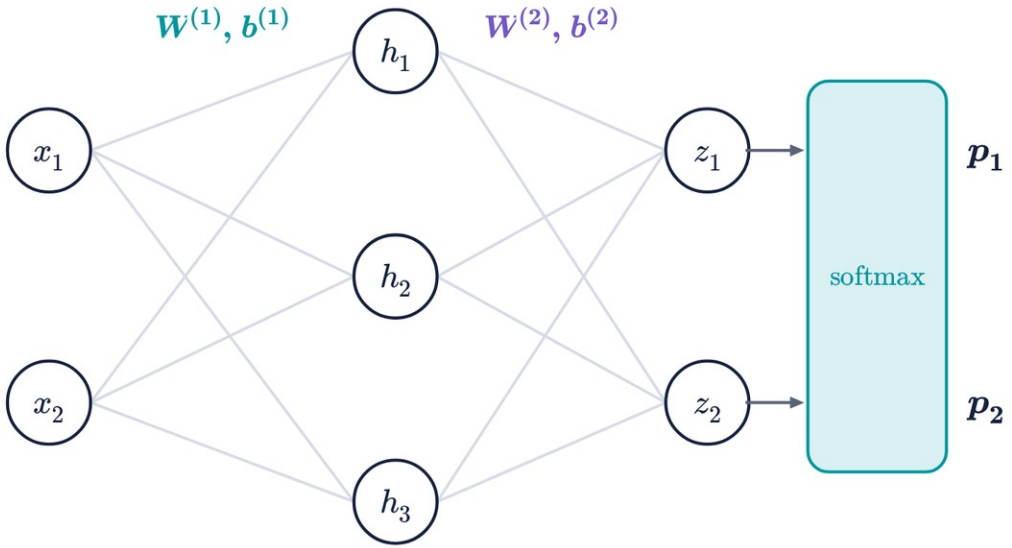
LAST TIME

Forward propagation

$$h = \sigma(W^{(1)}x + b^{(1)})$$
$$z = W^{(2)}h + b^{(2)}$$
$$p = \text{softmax}(z)$$

- Train by gradient descent:

  $\theta \leftarrow \theta - \eta \nabla \mathcal{L}(\theta)$



LAST TIME

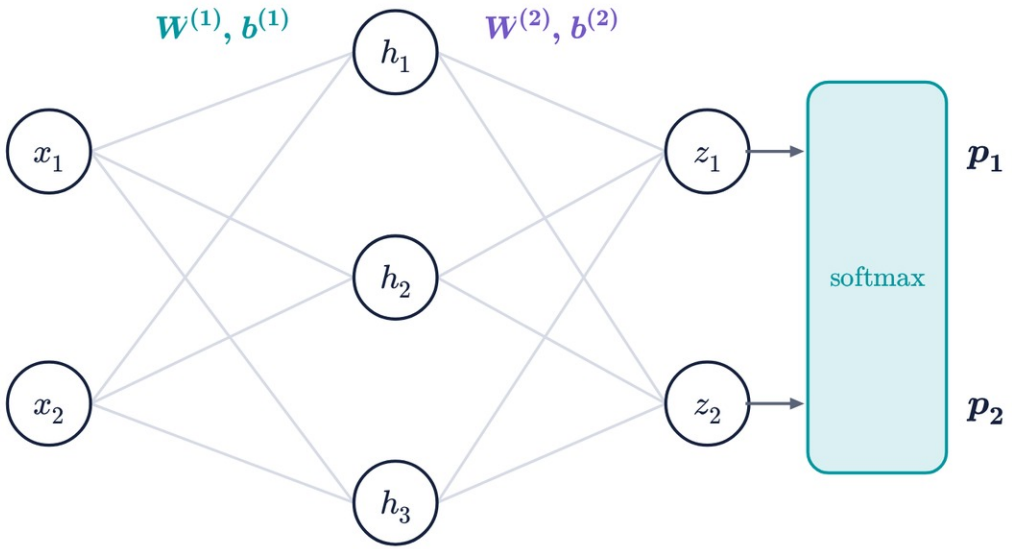
Forward propagation

$$h = \sigma(W^{(1)}x + b^{(1)})$$
$$z = W^{(2)}h + b^{(2)}$$
$$p = \text{softmax}(z)$$

- Train by gradient descent:

  $\theta \leftarrow \theta - \eta \nabla \mathcal{L}(\theta)$

- We need $\partial \mathcal{L}/\partial w$ for every weight w. Where does it come from?



BACKPROPAGATION

The network is one formula

A small network: two inputs, two hidden units, one output. Written out in full:

$$p = \sigma\Big(v_1\,\sigma(w_{11}x_1 + w_{12}x_2 + b_1) + v_2\,\sigma(w_{21}x_1 + w_{22}x_2 + b_2) + c\Big)$$

BACKPROPAGATION

The network is one formula

A small network: two inputs, two hidden units, one output. Written out in full:

$$p = \sigma\Big(v_1\,\sigma(w_{11}x_1 + w_{12}x_2 + b_1) + v_2\,\sigma(w_{21}x_1 + w_{22}x_2 + b_2) + c\Big)$$

The loss of one training point:

$$\mathcal{L} = -y \log p - (1-y)\log(1-p)$$

BACKPROPAGATION

The network is one formula

A small network: two inputs, two hidden units, one output. Written out in full:

$$p = \sigma\Big(v_1\,\sigma(w_{11}x_1 + w_{12}x_2 + b_1) + v_2\,\sigma(w_{21}x_1 + w_{22}x_2 + b_2) + c\Big)$$

The loss of one training point:

$$\mathcal{L} = -y \log p - (1-y)\log(1-p)$$

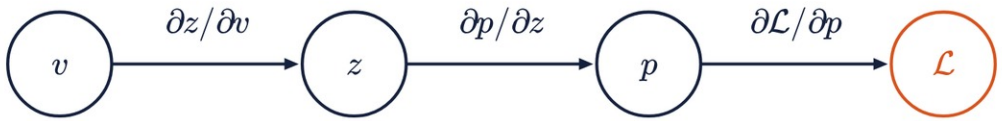
Nine weights, all inside one nested formula. We need $\partial\mathcal{L}/\partial w$ for each one.

BACKPROPAGATION

The chain rule

- A weight changes the loss through a chain of functions. Multiply the derivatives along the chain:

$$\frac{\partial \mathcal{L}}{\partial v} = \frac{\partial \mathcal{L}}{\partial p} \cdot \frac{\partial p}{\partial z} \cdot \frac{\partial z}{\partial v}$$



The chain rule

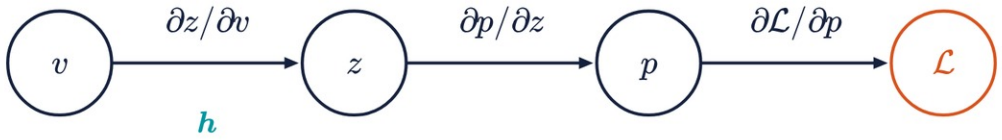
- A weight changes the loss through a chain of functions. Multiply the derivatives along the chain:

$$\frac{\partial \mathcal{L}}{\partial v} = \frac{\partial \mathcal{L}}{\partial p} \cdot \frac{\partial p}{\partial z} \cdot \frac{\partial z}{\partial v}$$

- One output neuron, label $y = 1$ or 0:

$$z = v_1 h_1 + v_2 h_2 + c, \quad p = \sigma(z)$$

$$\mathcal{L} = -y \log p - (1 - y) \log(1 - p)$$



The chain rule

- A weight changes the loss through a chain of functions. Multiply the derivatives along the chain:

$$\frac{\partial \mathcal{L}}{\partial v} = \frac{\partial \mathcal{L}}{\partial p} \cdot \frac{\partial p}{\partial z} \cdot \frac{\partial z}{\partial v}$$

- One output neuron, label $y = 1$ or 0:

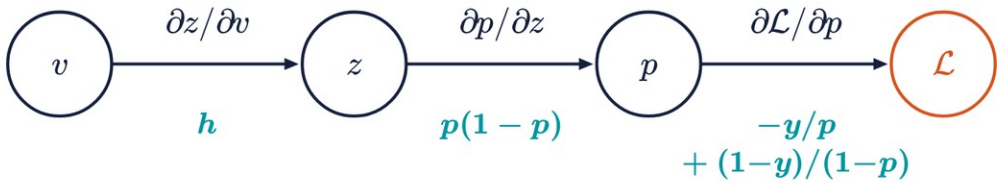
$$z = v_1 h_1 + v_2 h_2 + c, \quad p = \sigma(z)$$

$$\mathcal{L} = -y \log p - (1 - y) \log(1 - p)$$

- The two factors:

$$\frac{\partial \mathcal{L}}{\partial p} = -\frac{y}{p} + \frac{1 - y}{1 - p}$$

$$\frac{\partial p}{\partial z} = \sigma'(z) = p(1 - p)$$



The chain rule

- A weight changes the loss through a chain of functions. Multiply the derivatives along the chain:

  $$\frac{\partial \mathcal{L}}{\partial v} = \frac{\partial \mathcal{L}}{\partial p} \cdot \frac{\partial p}{\partial z} \cdot \frac{\partial z}{\partial v}$$

- One output neuron, label $y = 1$ or 0:

  $$z = v_1 h_1 + v_2 h_2 + c, \quad p = \sigma(z)$$

  $$\mathcal{L} = -y \log p - (1 - y) \log(1 - p)$$

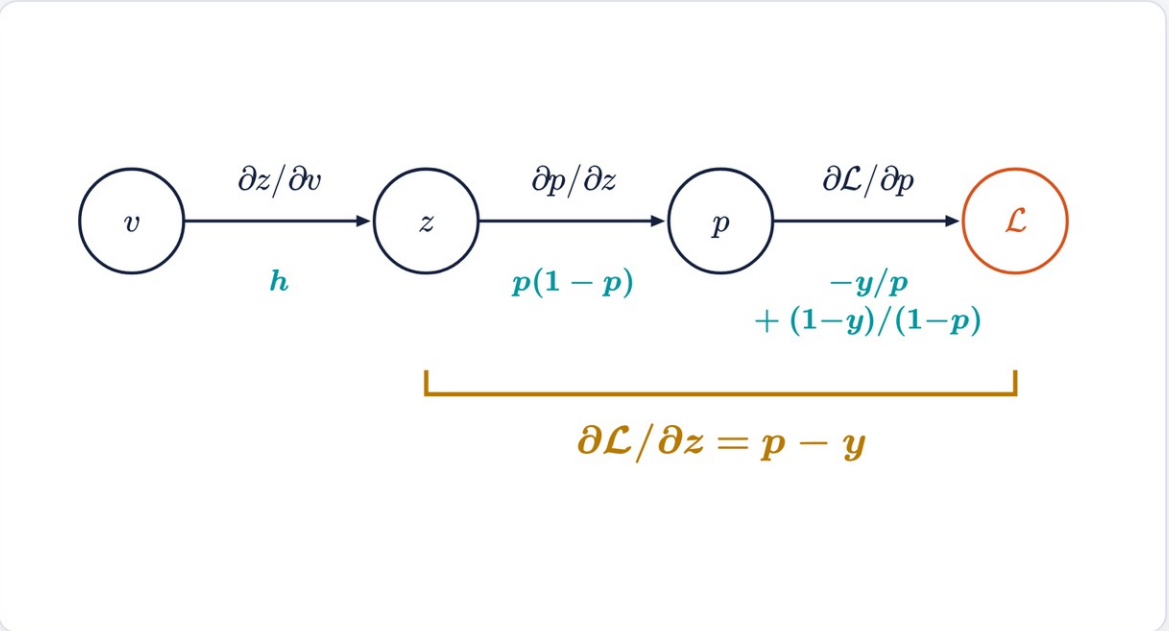
- The two factors:

  $$\frac{\partial \mathcal{L}}{\partial p} = -\frac{y}{p} + \frac{1 - y}{1 - p}$$

  $$\frac{\partial p}{\partial z} = \sigma'(z) = p(1 - p)$$

- Multiply them; it simplifies:

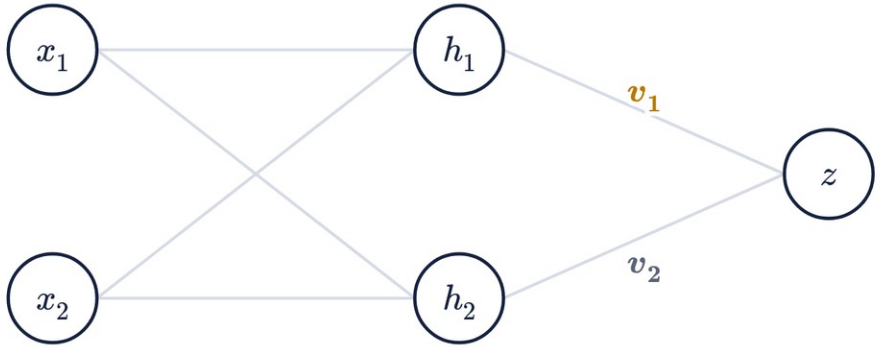
  $$\frac{\partial \mathcal{L}}{\partial z} = p - y$$



BACKPROPAGATION

Output-layer weights

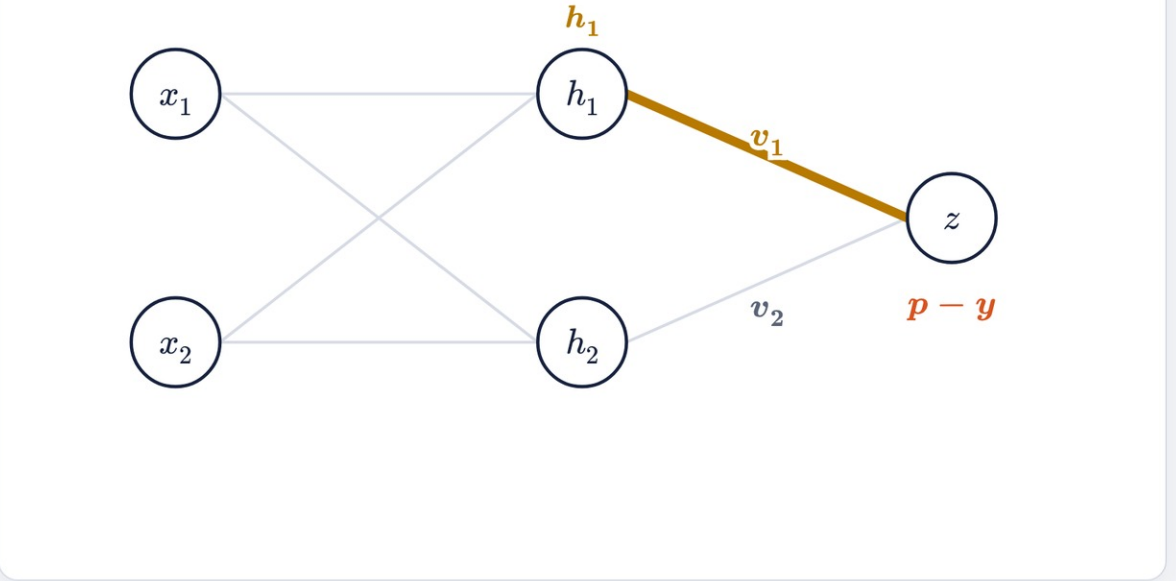
- A small network: two inputs, two hidden units, one output.



BACKPROPAGATION

Output-layer weights

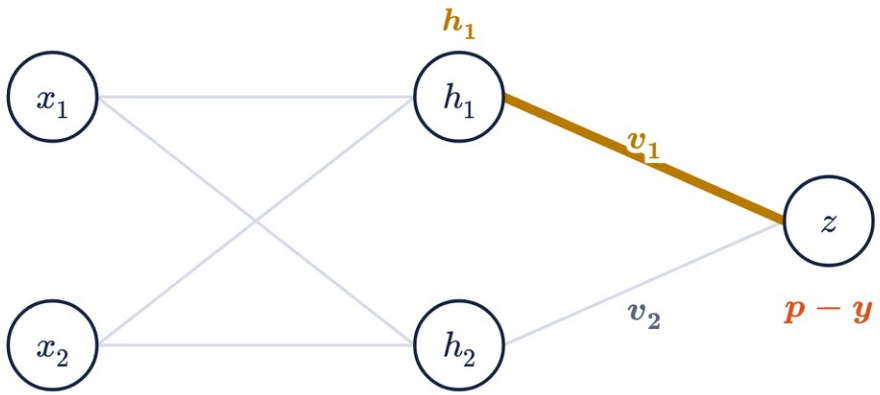
- A small network: two inputs, two hidden units, one output.
- The weight v_j reaches the loss through z, and $\partial z / \partial v_j = h_j$:
  $$\frac{\partial \mathcal{L}}{\partial v_j} = (p - y)\, h_j$$



BACKPROPAGATION

Output-layer weights

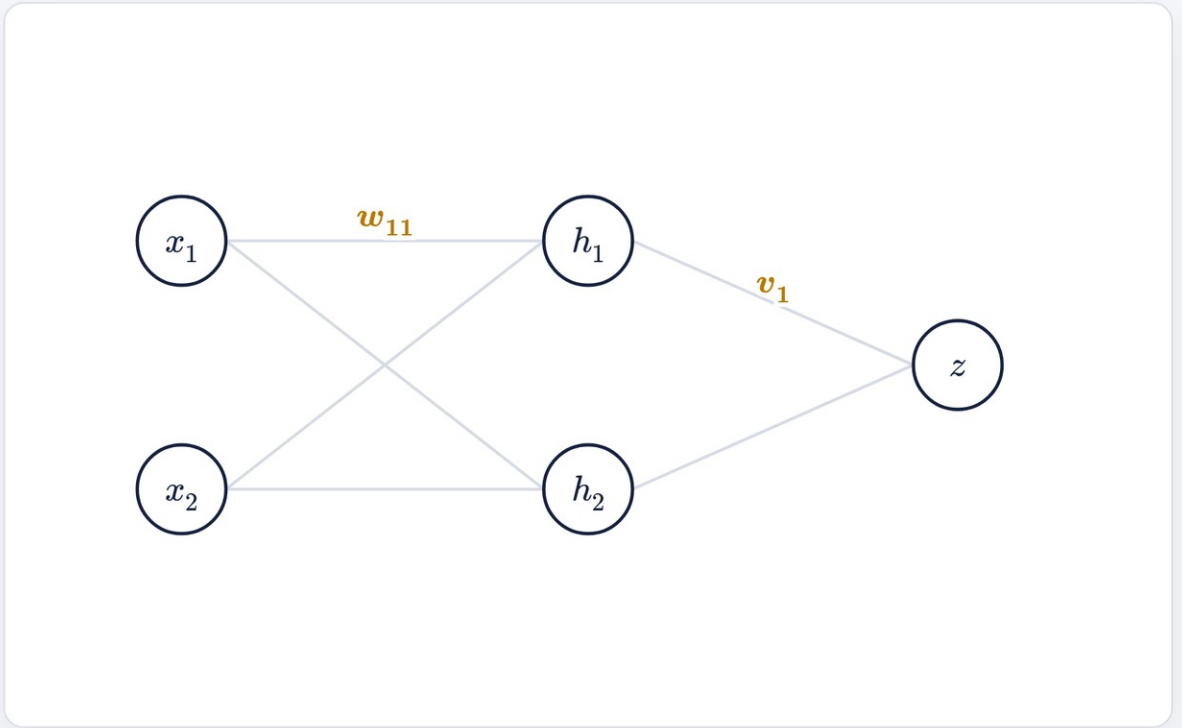
- A small network: two inputs, two hidden units, one output.
- The weight v_j reaches the loss through z, and $\partial z / \partial v_j = h_j$:
$$\frac{\partial \mathcal{L}}{\partial v_j} = (p - y)\, h_j$$
- The error at the output times the input to the weight.



BACKPROPAGATION

One layer deeper

- The weight w_{ji} connects input x_i to hidden unit $h_j = \sigma(a_j)$, with $a_j = w_{j1}x_1 + w_{j2}x_2 + b_j$.



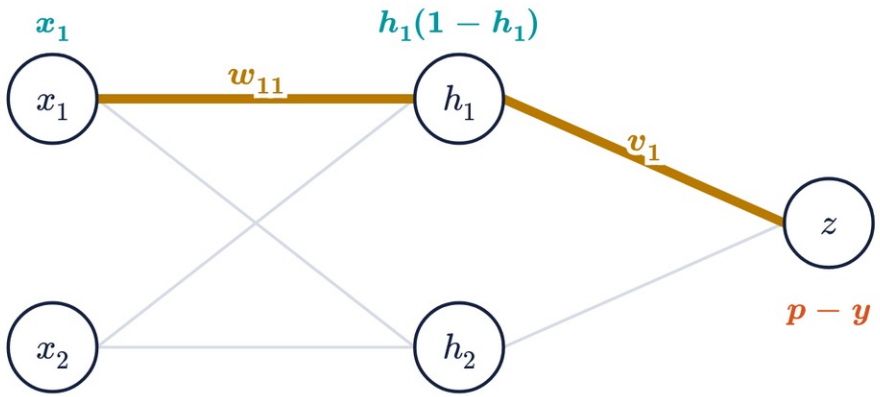
BACKPROPAGATION

One layer deeper

- The weight w_{ji} connects input x_i to hidden unit $h_j = \sigma(a_j)$, with $a_j = w_{j1}x_1 + w_{j2}x_2 + b_j$.
- Follow the chain from the loss back to w_{ji}:

$$\frac{\partial \mathcal{L}}{\partial w_{ji}} = (p - y) \cdot v_j \cdot h_j(1 - h_j) \cdot x_i$$

$h_j(1-h_j)$ is the sigmoid's slope $\sigma'(a_j)$. With another activation φ, use $\varphi'(a_j)$.



BACKPROPAGATION

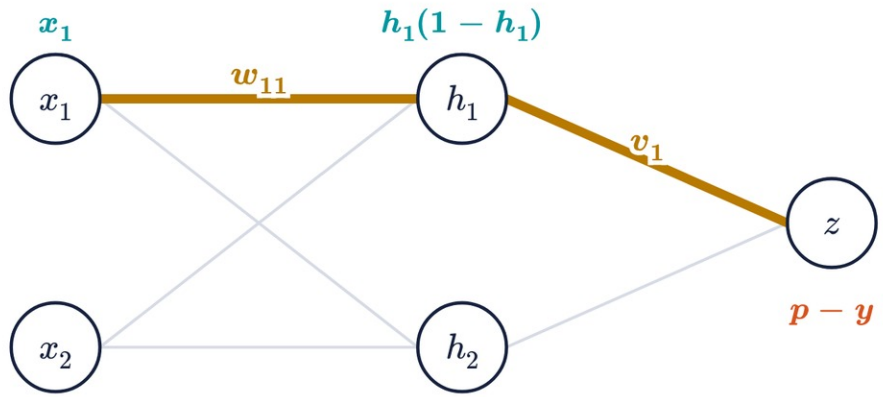
One layer deeper

- The weight w_{ji} connects input x_i to hidden unit $h_j = \sigma(a_j)$, with $a_j = w_{j1}x_1 + w_{j2}x_2 + b_j$.
- Follow the chain from the loss back to w_{ji}:

$$\frac{\partial \mathcal{L}}{\partial w_{ji}} = (p - y) \cdot v_j \cdot h_j(1 - h_j) \cdot x_i$$

  $h_j(1 - h_j)$ is the sigmoid's slope $\sigma'(a_j)$. With another activation φ, use $\varphi'(a_j)$.

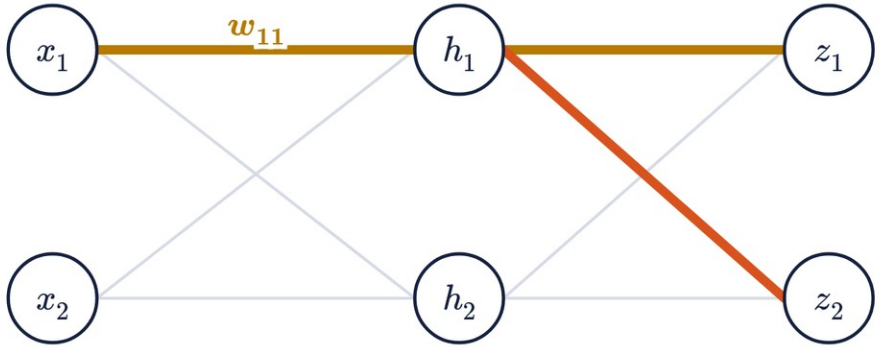
- The first factor, $p - y$, was already computed for the output layer. Reuse it.



BACKPROPAGATION

Several paths: add them

- With two outputs, h_j feeds both. A hidden weight reaches the loss along two paths.

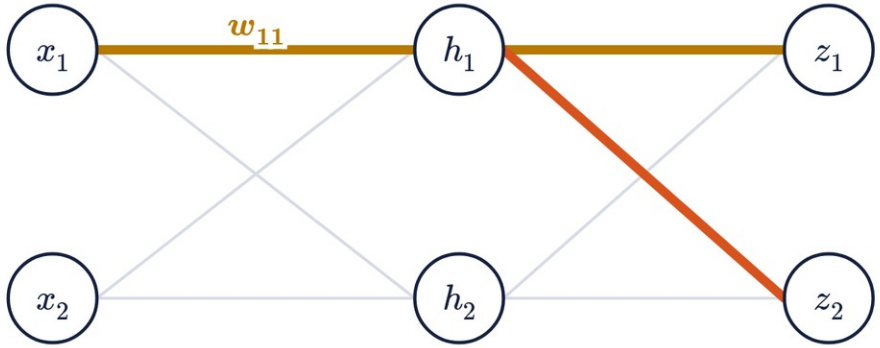


BACKPROPAGATION

Several paths: add them

- With two outputs, h_j feeds both. A hidden weight reaches the loss along two paths.
- Add the paths:

$$\frac{\partial \mathcal{L}}{\partial w_{ji}} = \sum_k (p_k - y_k)\, v_{kj} \cdot h_j(1 - h_j) \cdot x_i$$



BACKPROPAGATION

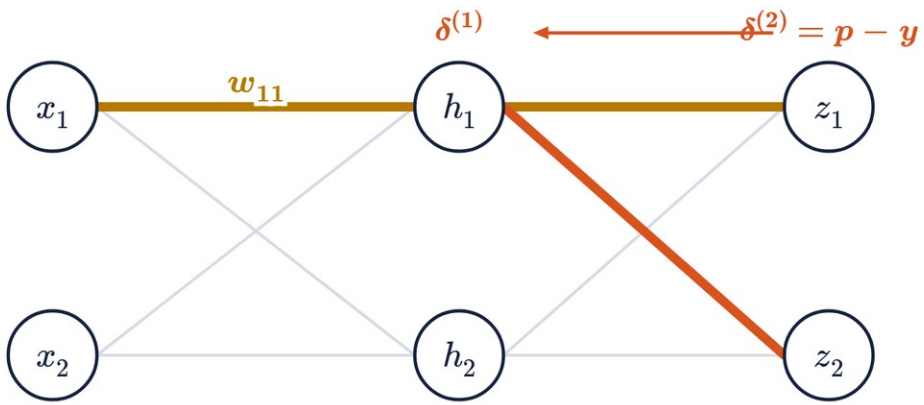
Several paths: add them

- With two outputs, h_j feeds both. A hidden weight reaches the loss along two paths.
- Add the paths:

$$\frac{\partial \mathcal{L}}{\partial w_{ji}} = \sum_k (p_k - y_k)\, v_{kj} \cdot h_j(1 - h_j) \cdot x_i$$

- In matrix form, with $\odot$ the entrywise product:

$$\delta^{(2)} = p - y, \quad \delta^{(1)} = (W^{(2)\top} \delta^{(2)}) \odot h \odot (1 - h)$$



BACKPROPAGATION

Several paths: add them

- With two outputs, h_j feeds both. A hidden weight reaches the loss along two paths.
- Add the paths:

$$\frac{\partial \mathcal{L}}{\partial w_{ji}} = \sum_k (p_k - y_k)\, v_{kj} \cdot h_j(1 - h_j) \cdot x_i$$

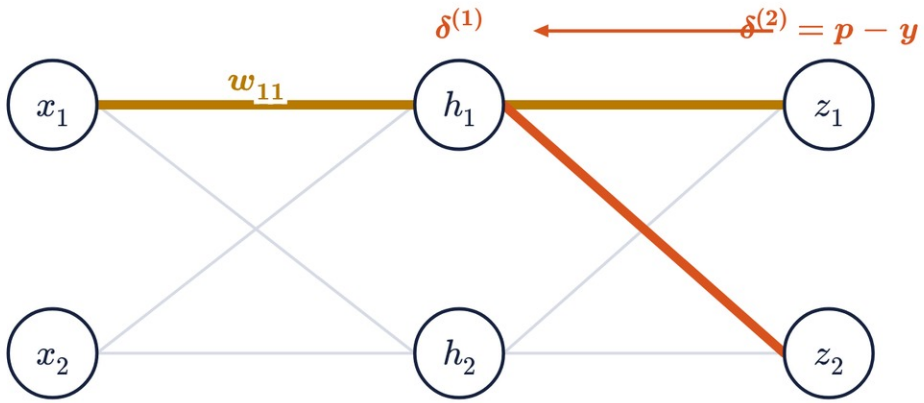
- In matrix form, with $\odot$ the entrywise product:

$$\delta^{(2)} = p - y, \quad \delta^{(1)} = (W^{(2)\top} \delta^{(2)}) \odot h \odot (1 - h)$$

- Then:

$$\frac{\partial \mathcal{L}}{\partial W^{(2)}} = \delta^{(2)} h^\top, \quad \frac{\partial \mathcal{L}}{\partial W^{(1)}} = \delta^{(1)} x^\top$$

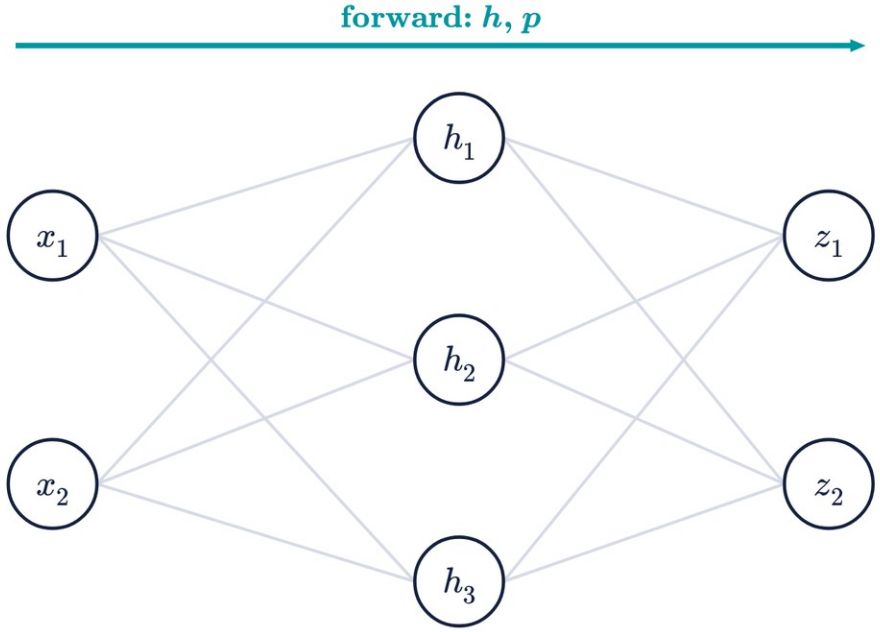
The biases get $\delta^{(2)}$ and $\delta^{(1)}$.



BACKPROPAGATION

The recipe

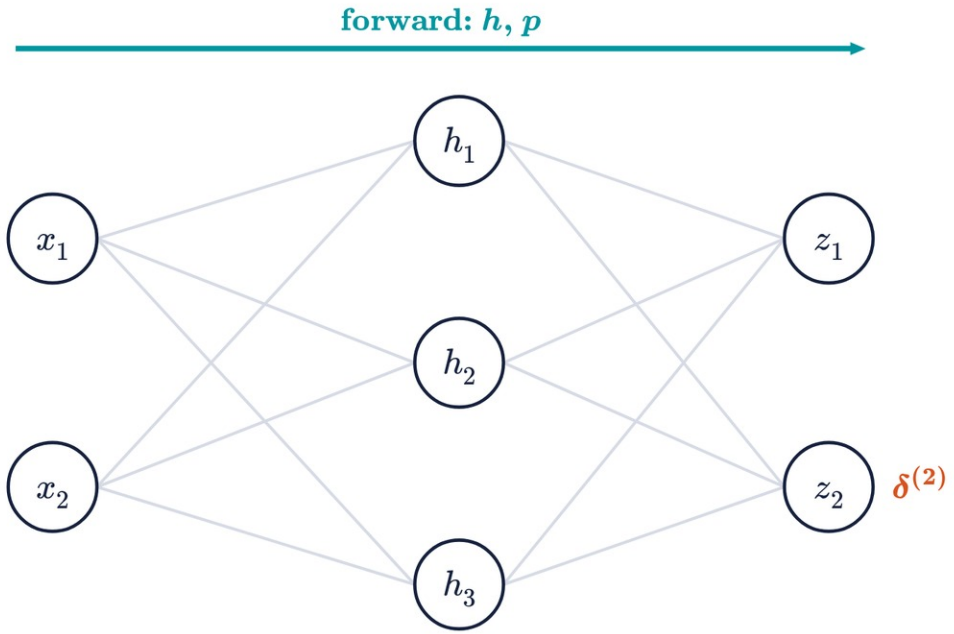
1. **Forward pass**: compute and store h and p.



BACKPROPAGATION

The recipe

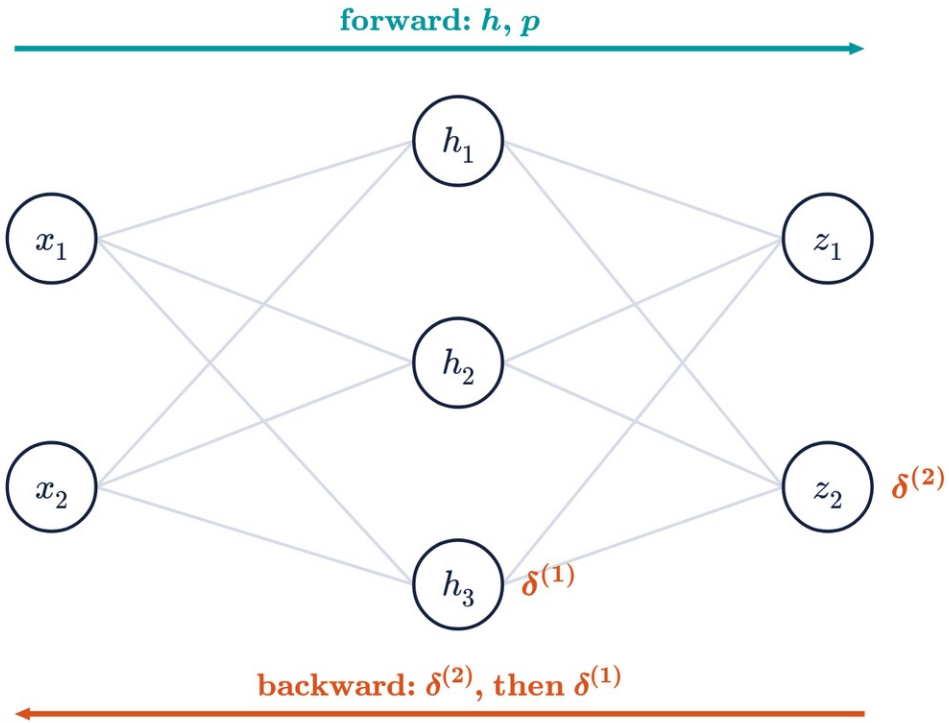
1. **Forward pass**: compute and store h and p.
2. **Error at the output**: $\delta^{(2)} = p - y$.



BACKPROPAGATION

The recipe

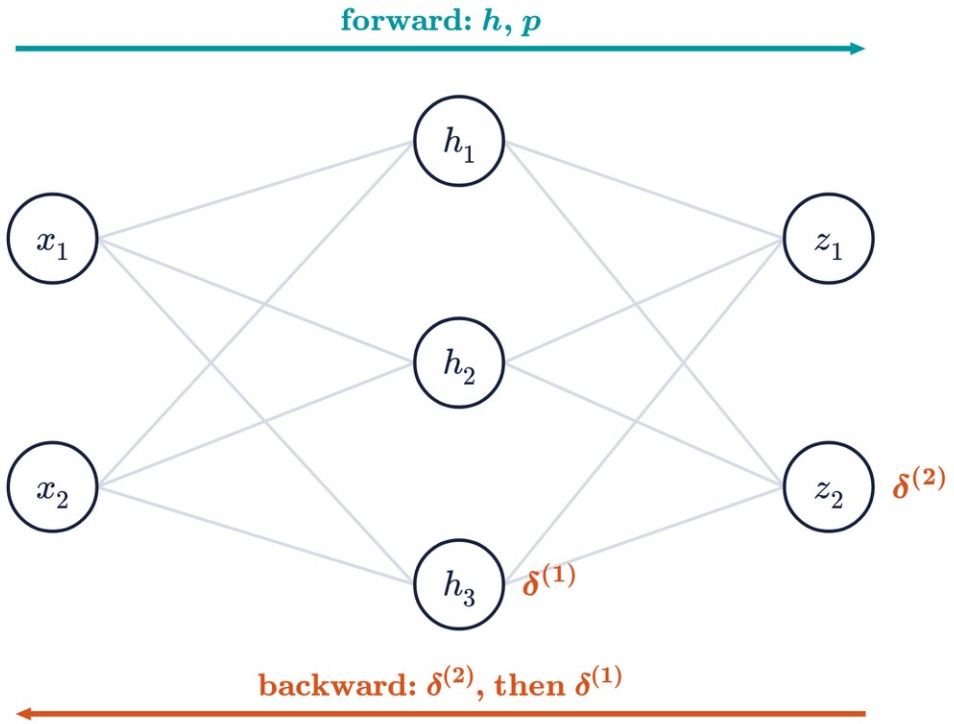
1. **Forward pass**: compute and store h and p.
2. **Error at the output**: $\delta^{(2)} = p - y$.
3. **Backward pass**:
   $\delta^{(1)} = (W^{(2)\top}\delta^{(2)}) \odot h \odot (1 - h)$.
   For sigmoid units. With another activation, replace $h \odot (1 - h)$ by $\varphi'(a)$.



BACKPROPAGATION

The recipe

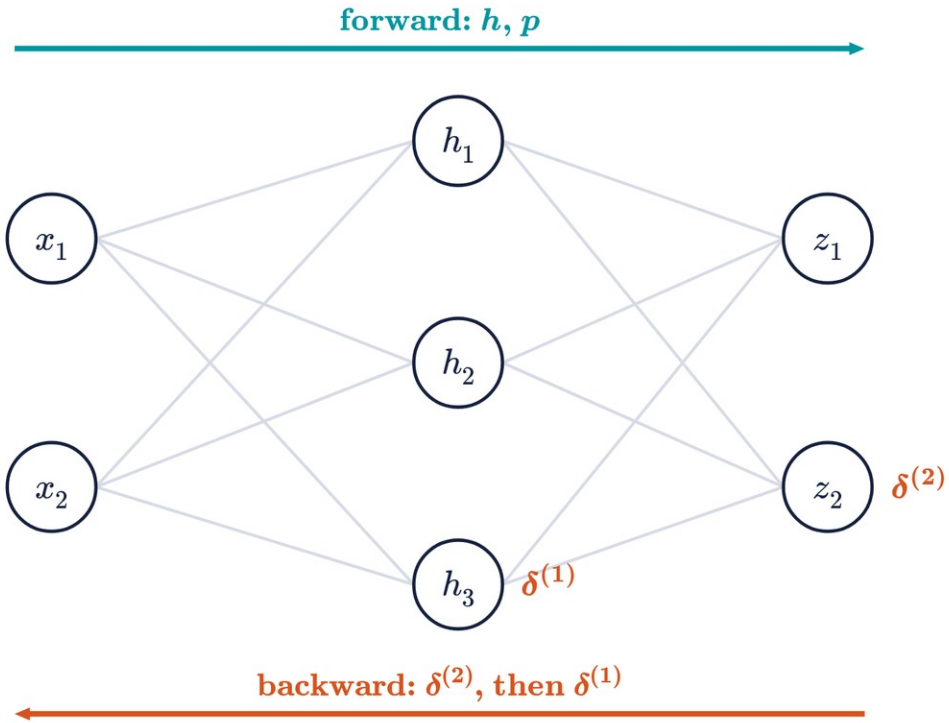
1. **Forward pass**: compute and store h and p.
2. **Error at the output**: $\delta^{(2)} = p - y$.
3. **Backward pass**:
   $\delta^{(1)} = (W^{(2)\top}\delta^{(2)}) \odot h \odot (1 - h)$.
   For sigmoid units. With another activation, replace $h \odot (1 - h)$ by $\varphi'(a)$.
4. **Gradients**: $\delta^{(2)}h^\top$, $\delta^{(1)}x^\top$, and the δs for the biases.



BACKPROPAGATION

The recipe

1. **Forward pass**: compute and store h and p.
2. **Error at the output**: $\delta^{(2)} = p - y$.
3. **Backward pass**:
   $\delta^{(1)} = (W^{(2)\top}\delta^{(2)}) \odot h \odot (1 - h)$.
   For sigmoid units. With another activation, replace $h \odot (1 - h)$ by $\varphi'(a)$.
4. **Gradients**: $\delta^{(2)}h^\top$, $\delta^{(1)}x^\top$, and the δs for the biases.
5. **Update** every weight, $\theta \leftarrow \theta - \eta \nabla \mathcal{L}$, using the old weights throughout.



PUTTING IT TOGETHER

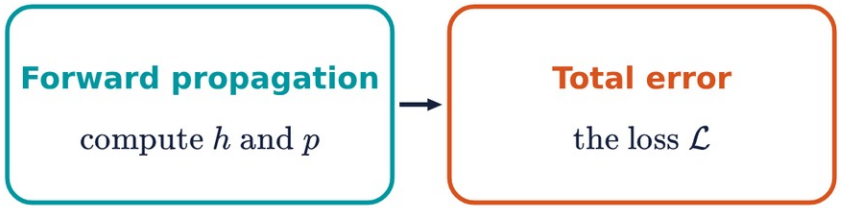
The training loop

Forward propagation

compute h and p

PUTTING IT TOGETHER

The training loop



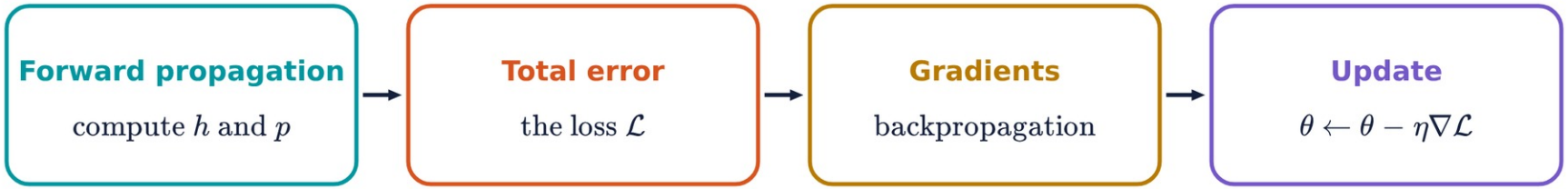
PUTTING IT TOGETHER

The training loop



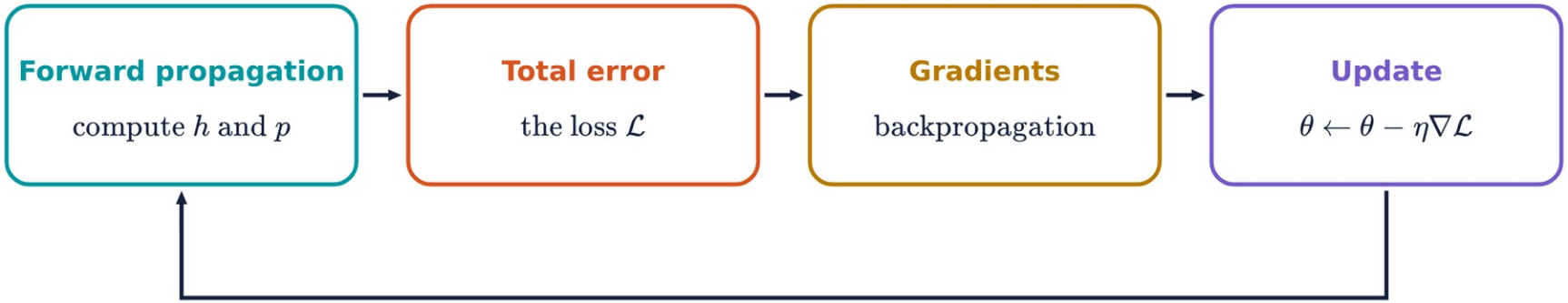
PUTTING IT TOGETHER

The training loop



PUTTING IT TOGETHER

The training loop



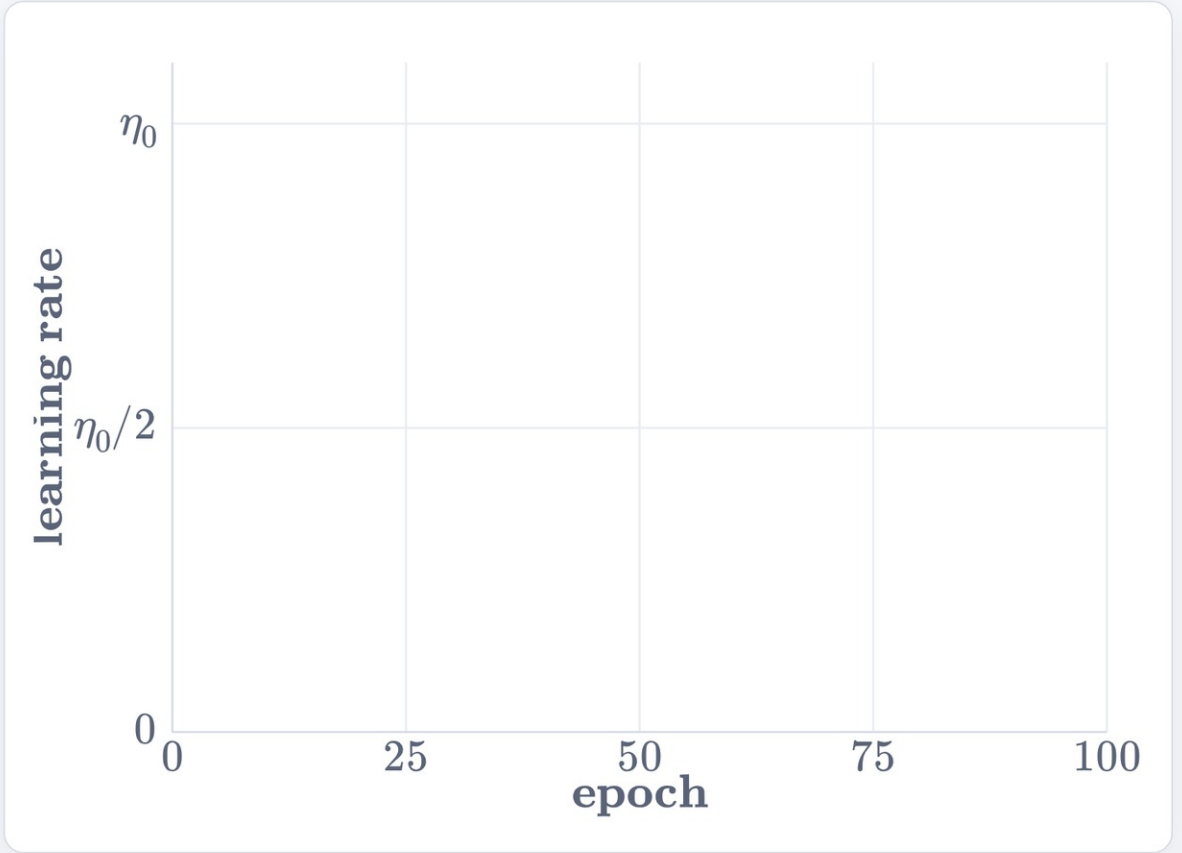
TRAINING IN PRACTICE

The learning rate

- The update:

  $\theta \leftarrow \theta - \eta \nabla \mathcal{L}(\theta)$

  Too small: slow. Too large: it overshoots. (Last class.)



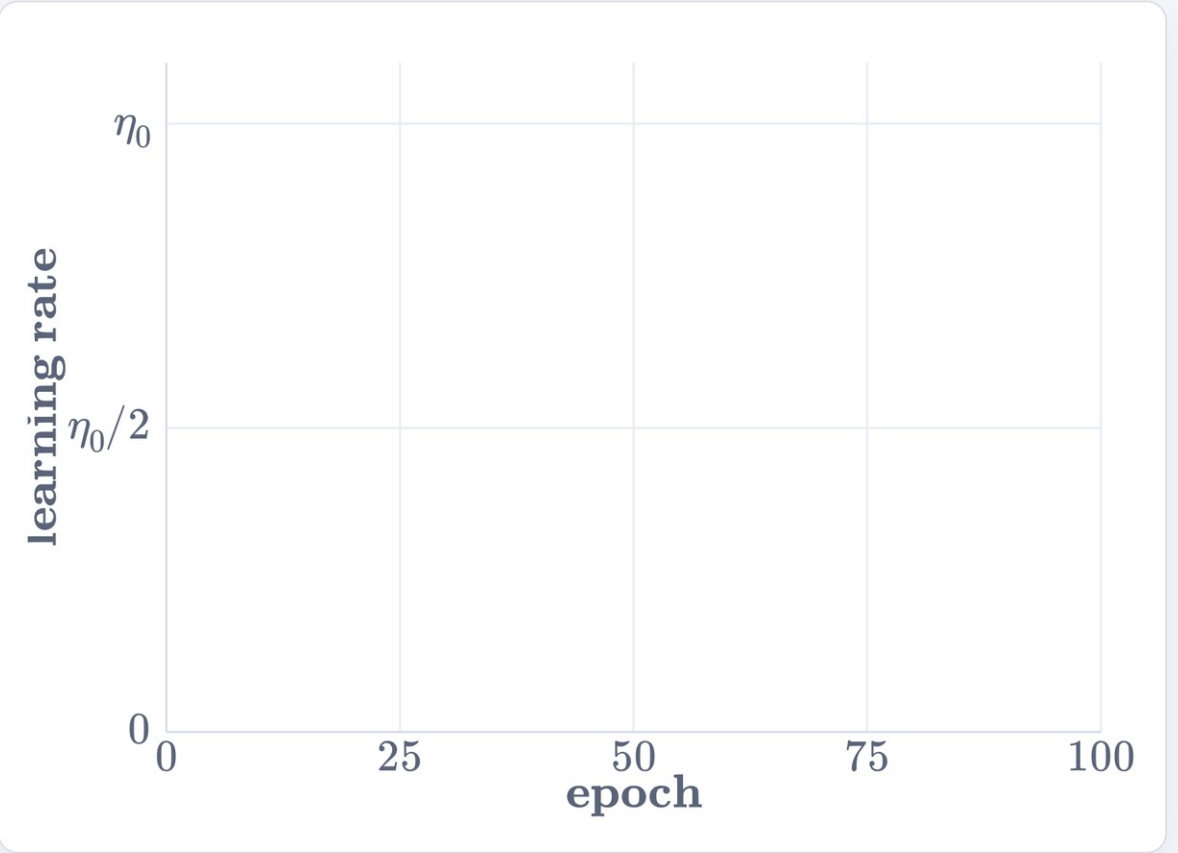
TRAINING IN PRACTICE

The learning rate

- The update:

  $\theta \leftarrow \theta - \eta \nabla \mathcal{L}(\theta)$

  Too small: slow. Too large: it overshoots. (Last class.)
- Large steps help early; small steps help near the end.



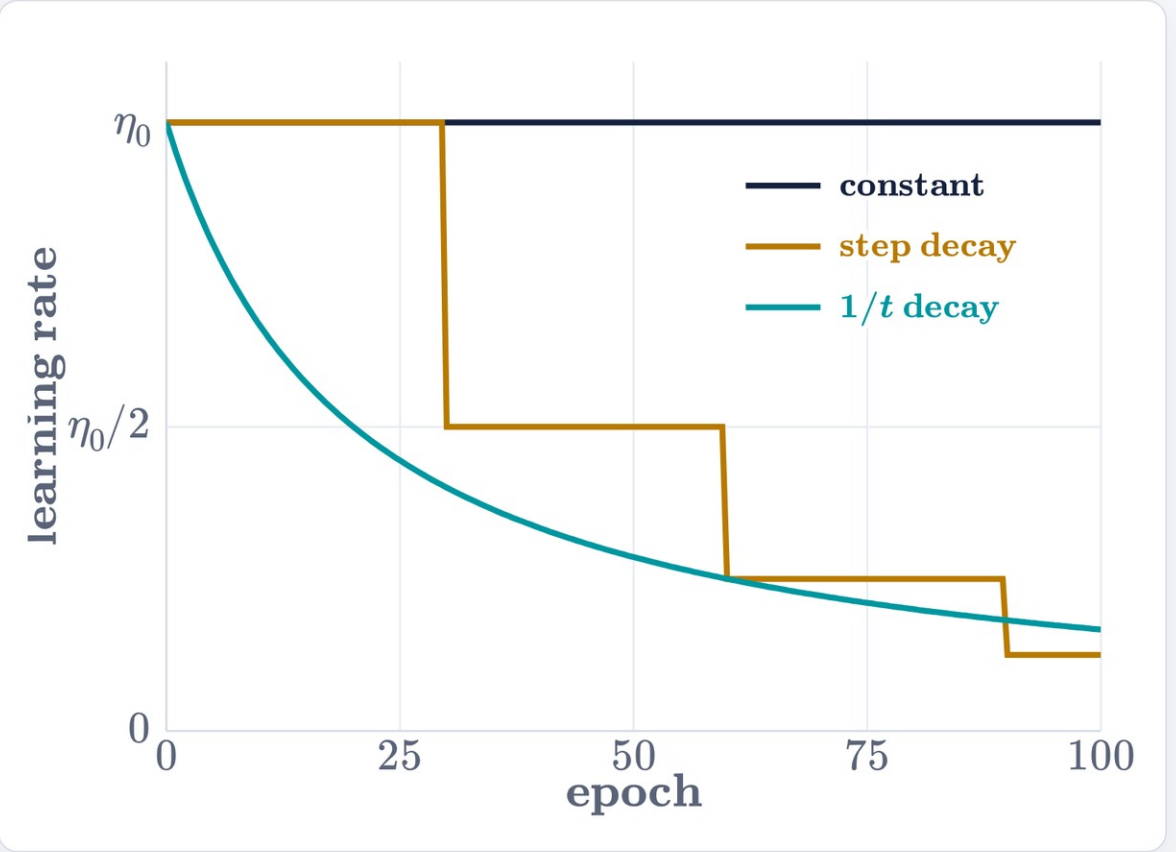
TRAINING IN PRACTICE

The learning rate

- The update:

  $\theta \leftarrow \theta - \eta \nabla \mathcal{L}(\theta)$

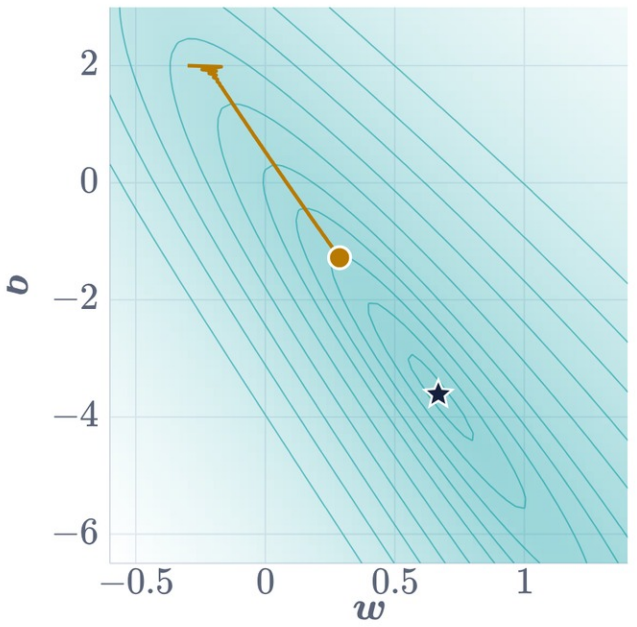
  Too small: slow. Too large: it overshoots. (Last class.)
- Large steps help early; small steps help near the end.
- A **learning-rate schedule** lowers η as training goes on.

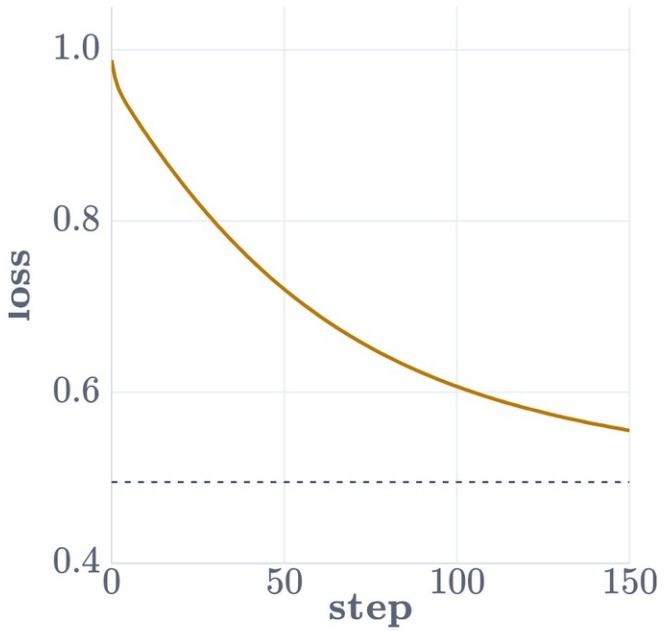


TRAINING IN PRACTICE

Momentum

- A long, narrow valley: plain gradient descent zigzags across it.





STEP
150

PLAIN LOSS
0.555

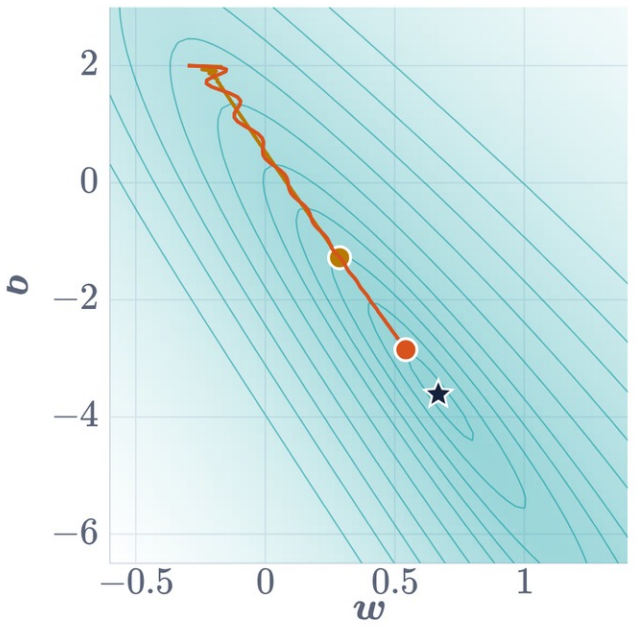
▶ step

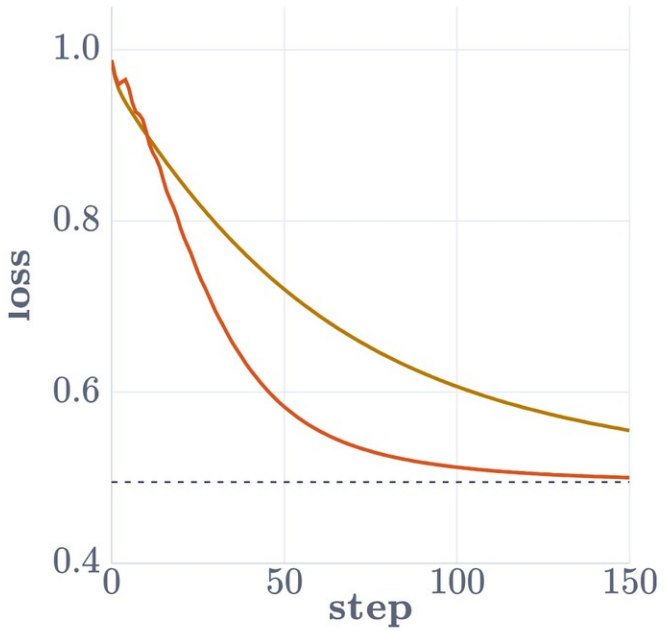
TRAINING IN PRACTICE

Momentum

- A long, narrow valley: plain gradient descent zigzags across it.
- Keep a running sum of past gradients:

  $v \leftarrow \beta v + \nabla \mathcal{L}, \quad \theta \leftarrow \theta - \eta\, v$





STEP	PLAIN LOSS	MOMENTUM LOSS
150	0.555	0.500

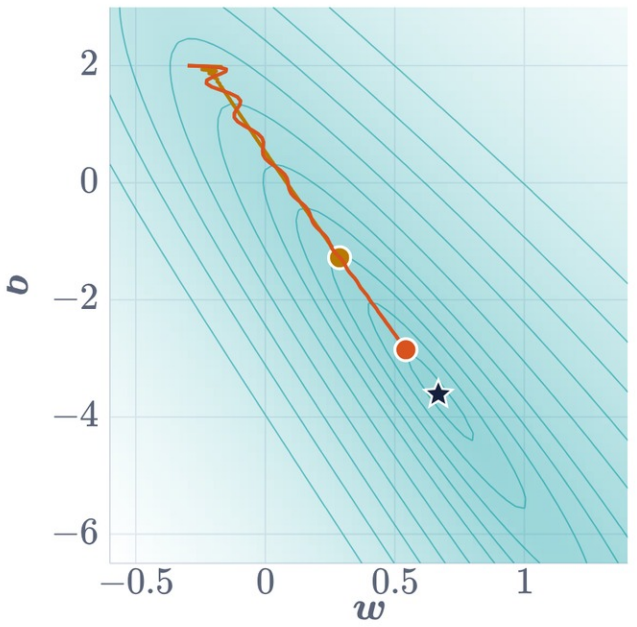
▶ step

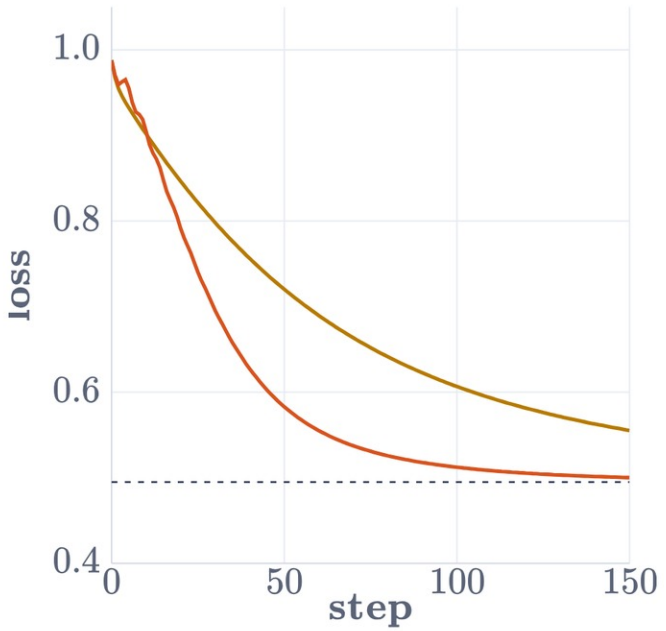
TRAINING IN PRACTICE

Momentum

- A long, narrow valley: plain gradient descent zigzags across it.
- Keep a running sum of past gradients:

  $v \leftarrow \beta v + \nabla \mathcal{L}, \quad \theta \leftarrow \theta - \eta\, v$
- The zigzags cancel; the steady downhill part adds up.





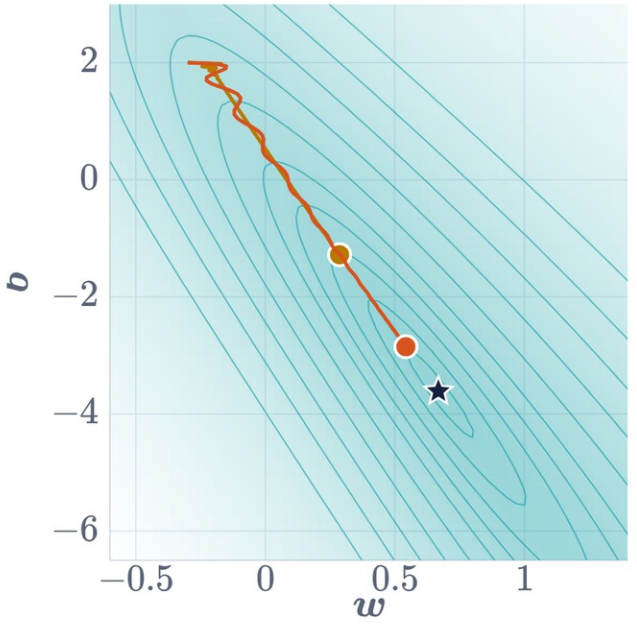
STEP	PLAIN LOSS	MOMENTUM LOSS
150	0.555	0.500

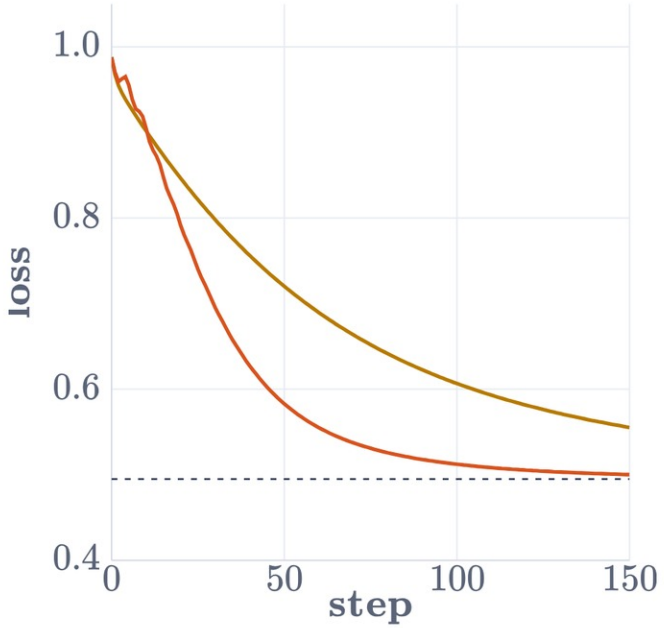
▶ step

TRAINING IN PRACTICE

Adam

- Momentum, plus a separate step size for each parameter.





PLAIN LOSS	MOMENTUM LOSS	ADAM LOSS
0.555	0.500	0.495

▶ step

TRAINING IN PRACTICE

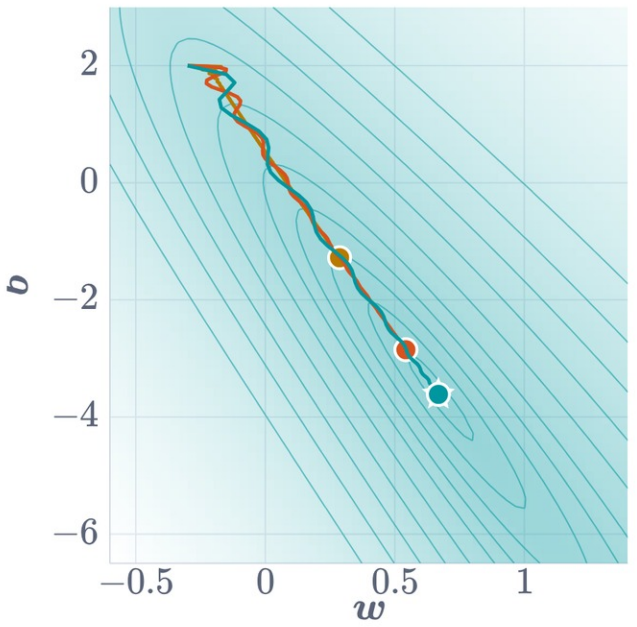
Adam

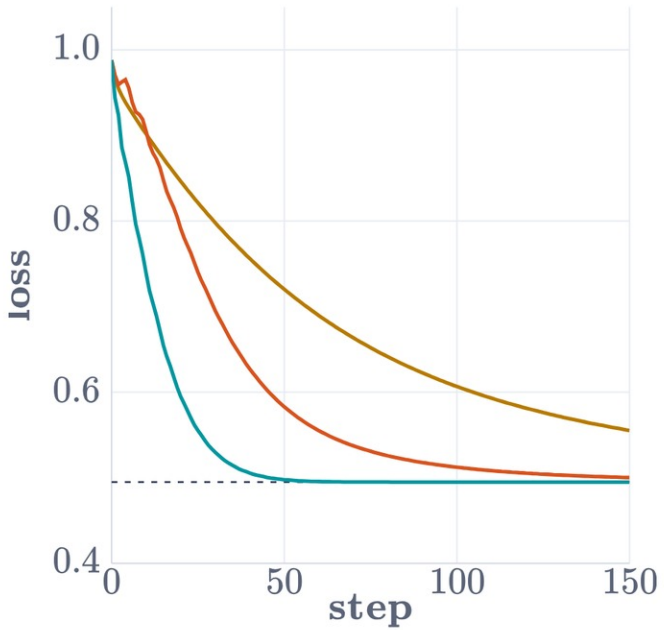
- Momentum, plus a separate step size for each parameter.
- Divide each parameter's step by a running average of its squared gradients:

$$m \leftarrow \beta_1 m + (1 - \beta_1)\,\nabla\mathcal{L}$$

$$s \leftarrow \beta_2 s + (1 - \beta_2)\,(\nabla\mathcal{L})^2$$

$$\theta \leftarrow \theta - \eta\,\frac{m}{\sqrt{s}}$$





PLAIN LOSS
0.555

MOMENTUM LOSS
0.500

ADAM LOSS
0.495

▶ step

TRAINING IN PRACTICE

Adam

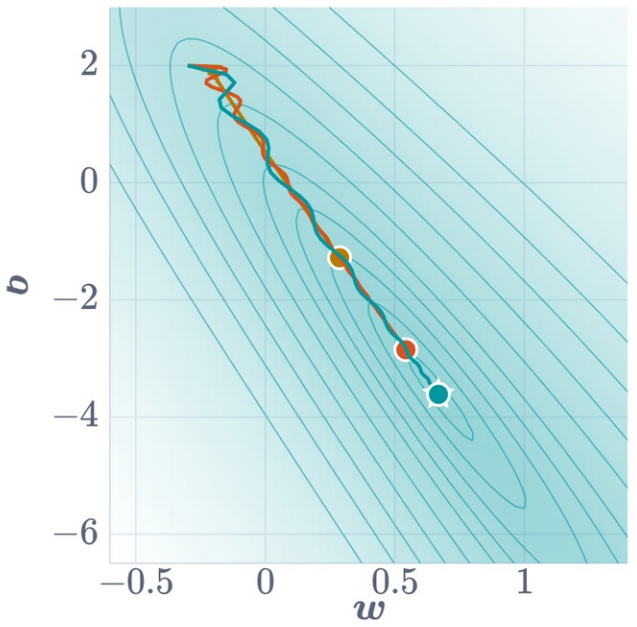
- Momentum, plus a separate step size for each parameter.
- Divide each parameter's step by a running average of its squared gradients:

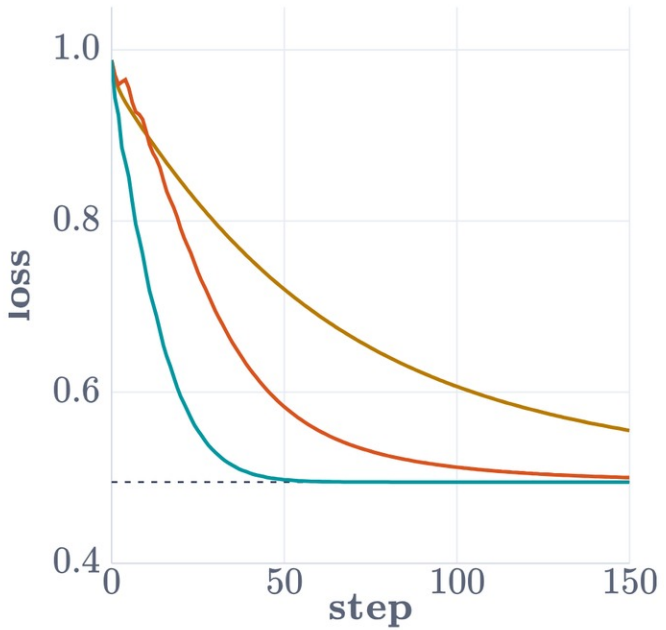
$$m \leftarrow \beta_1 m + (1 - \beta_1)\,\nabla\mathcal{L}$$

$$s \leftarrow \beta_2 s + (1 - \beta_2)\,(\nabla\mathcal{L})^2$$

$$\theta \leftarrow \theta - \eta\,\frac{m}{\sqrt{s}}$$

- The usual default in practice.





PLAIN LOSS
0.555

MOMENTUM LOSS
0.500

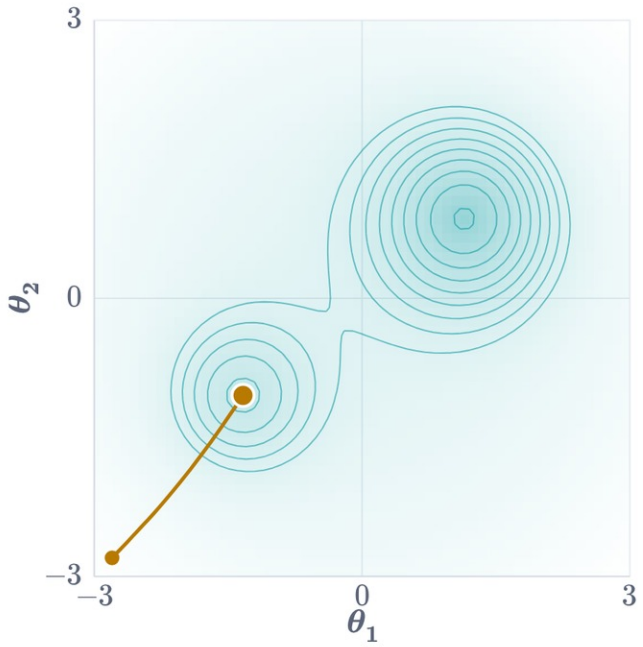
ADAM LOSS
0.495

▶ step

TRAINING IN PRACTICE

Local minima

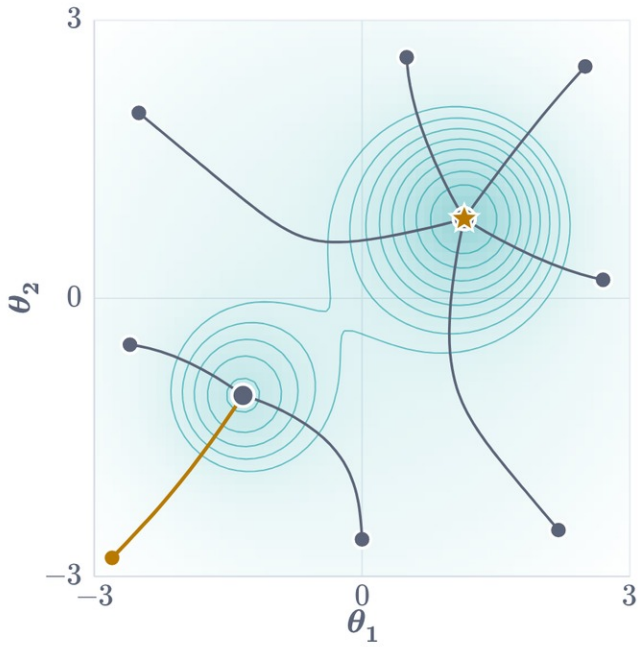
- Gradient descent stops in the valley it starts in.



TRAINING IN PRACTICE

Local minima

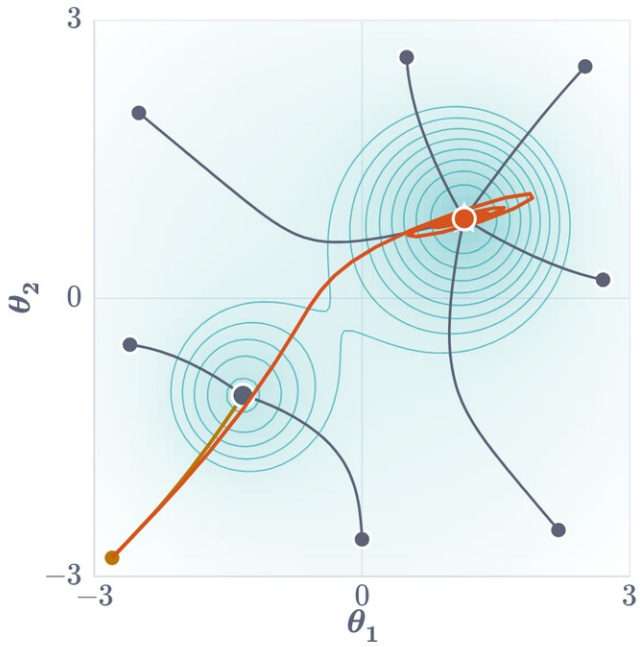
- Gradient descent stops in the valley it starts in.
- Restart from several starting points; keep the lowest loss.



TRAINING IN PRACTICE

Local minima

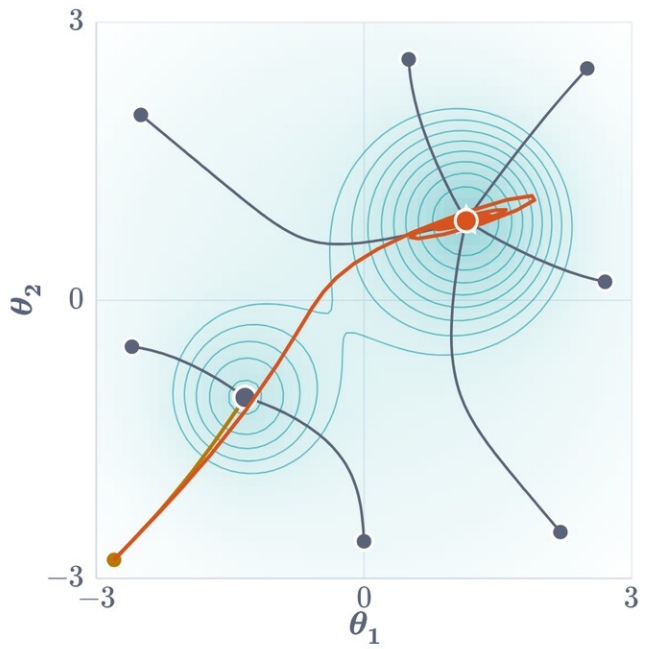
- Gradient descent stops in the valley it starts in.
- Restart from several starting points; keep the lowest loss.
- Momentum can roll through a shallow valley.

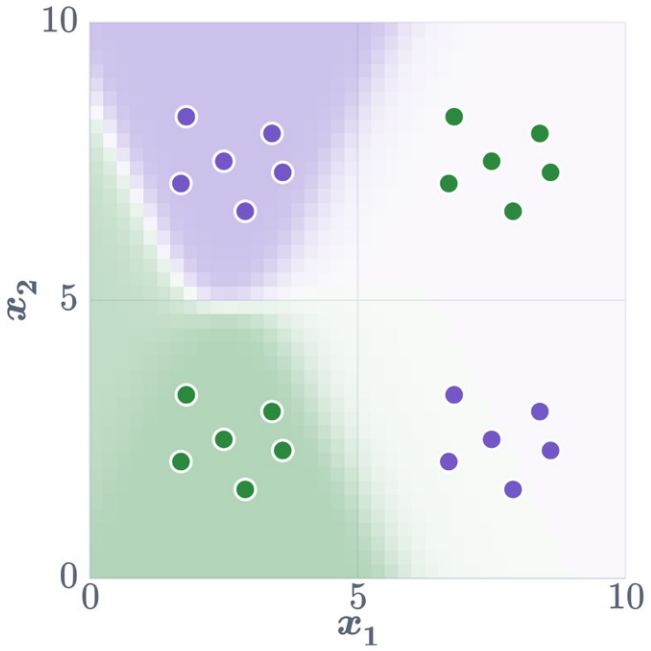


TRAINING IN PRACTICE

Local minima

- Gradient descent stops in the valley it starts in.
- Restart from several starting points; keep the lowest loss.
- Momentum can roll through a shallow valley.
- It happens on real networks: the XOR network from a different start stalls.



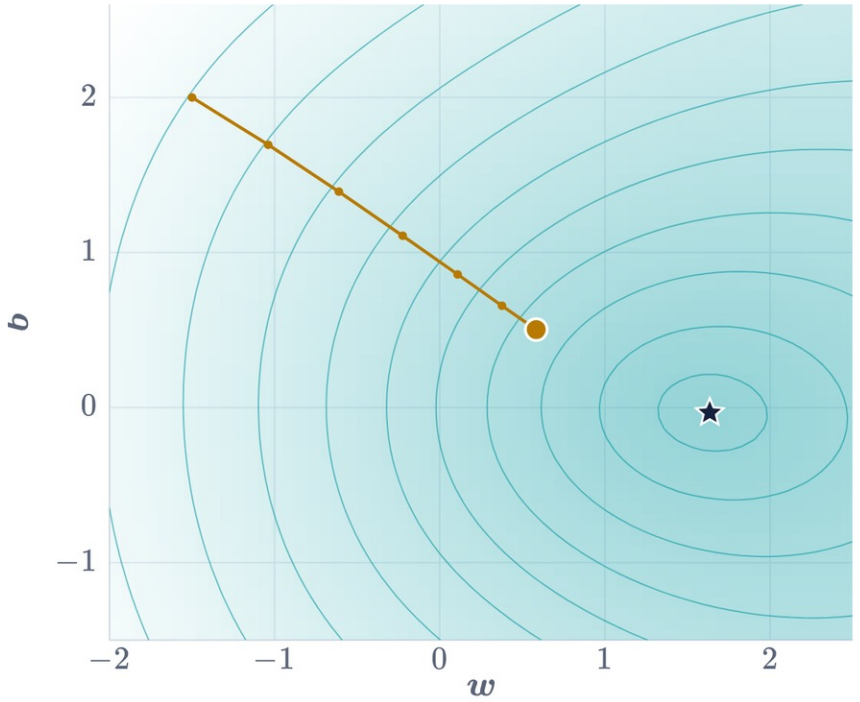


STEPS	LOSS	ACCURACY
4000	0.347	58%

TRAINING IN PRACTICE

How much data per step?

- **Batch**: the gradient from all the training points. One step per pass.



POINTS PER STEP
20

STEPS PER EPOCH
1

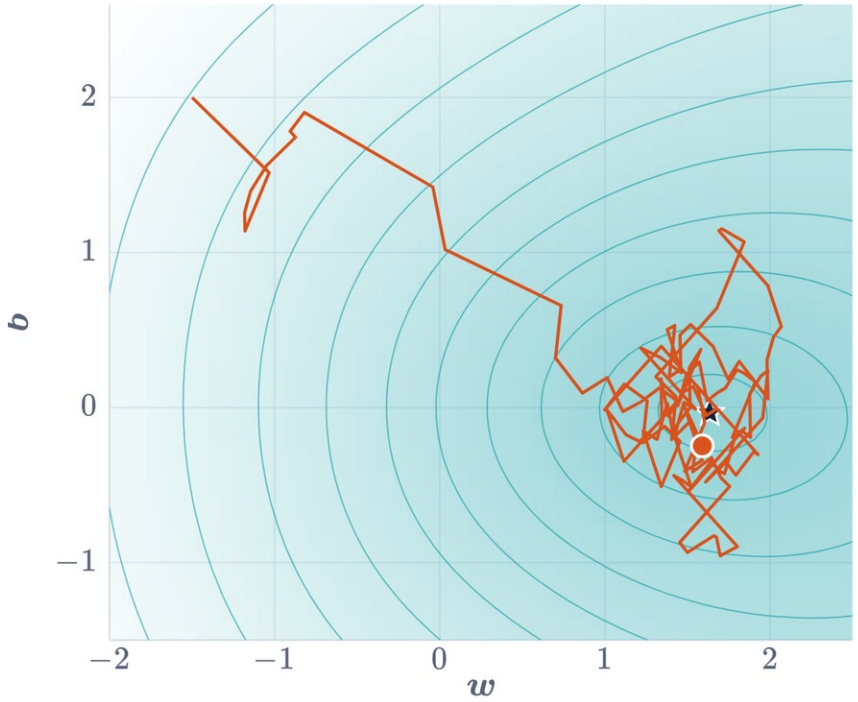
LOSS AFTER 6 EPOCHS
0.59

batch | stochastic | mini-batch | ▶

TRAINING IN PRACTICE

How much data per step?

- **Batch**: the gradient from all the training points. One step per pass.
- **Stochastic**: from one point at a time. Many cheap, noisy steps.



POINTS PER STEP
1

STEPS PER EPOCH
20

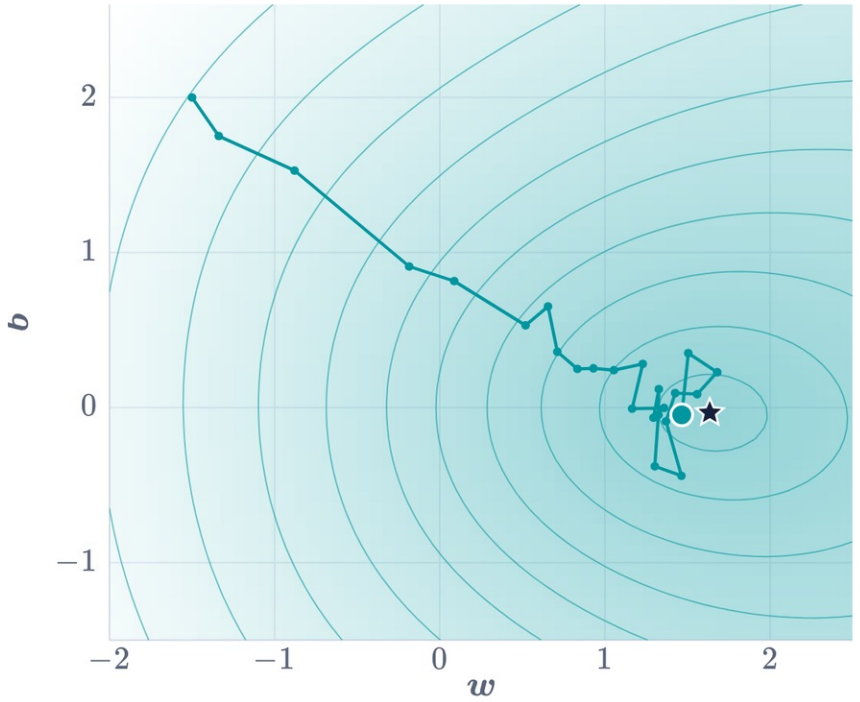
LOSS AFTER 6 EPOCHS
0.50

batch | stochastic | mini-batch | ▶

TRAINING IN PRACTICE

How much data per step?

- **Batch**: the gradient from all the training points. One step per pass.
- **Stochastic**: from one point at a time. Many cheap, noisy steps.
- **Mini-batch**: from a few points, here 5. In between.



POINTS PER STEP
5

STEPS PER EPOCH
4

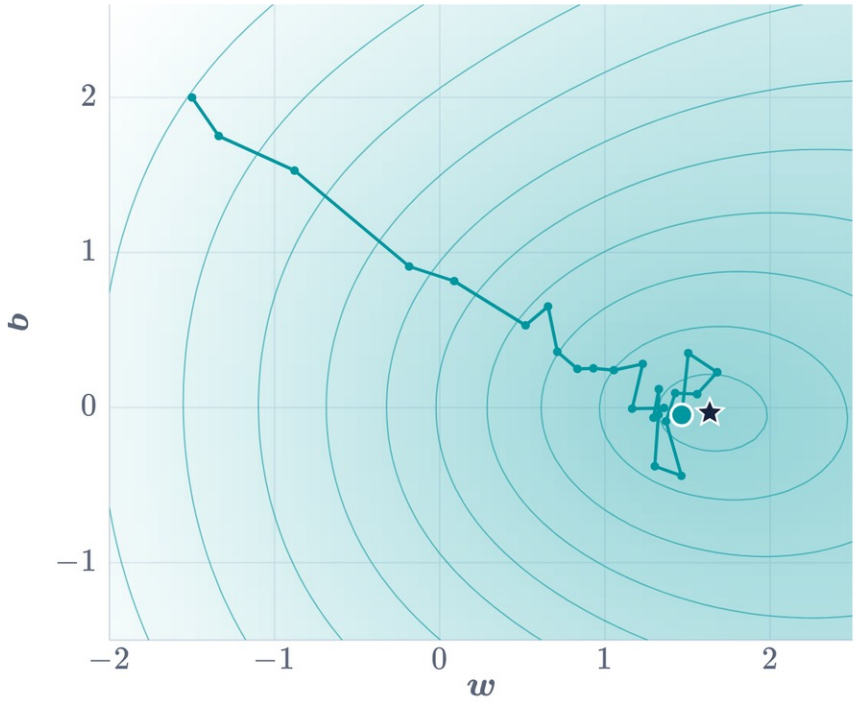
LOSS AFTER 6 EPOCHS
0.50

batch | stochastic | mini-batch | ▶

TRAINING IN PRACTICE

How much data per step?

- **Batch**: the gradient from all the training points. One step per pass.
- **Stochastic**: from one point at a time. Many cheap, noisy steps.
- **Mini-batch**: from a few points, here 5. In between.
- An **epoch** is one pass through the training data.



POINTS PER STEP
5

STEPS PER EPOCH
4

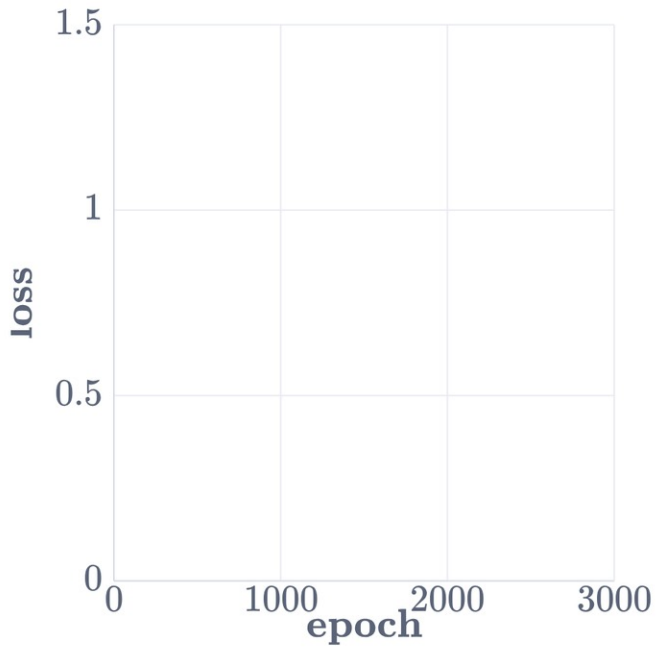
LOSS AFTER 6 EPOCHS
0.50

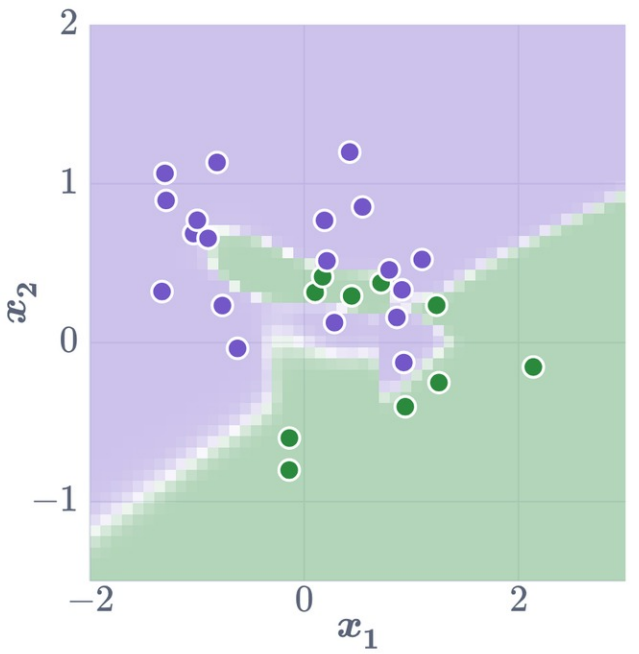
batch | stochastic | mini-batch | ▶

TRAINING IN PRACTICE

When to stop

- A maximum number of epochs; the weights stop changing; the loss is low enough.



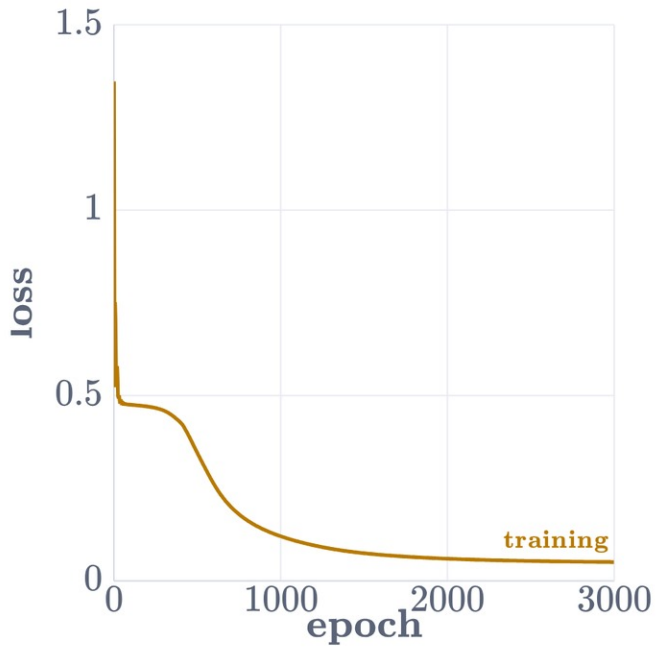


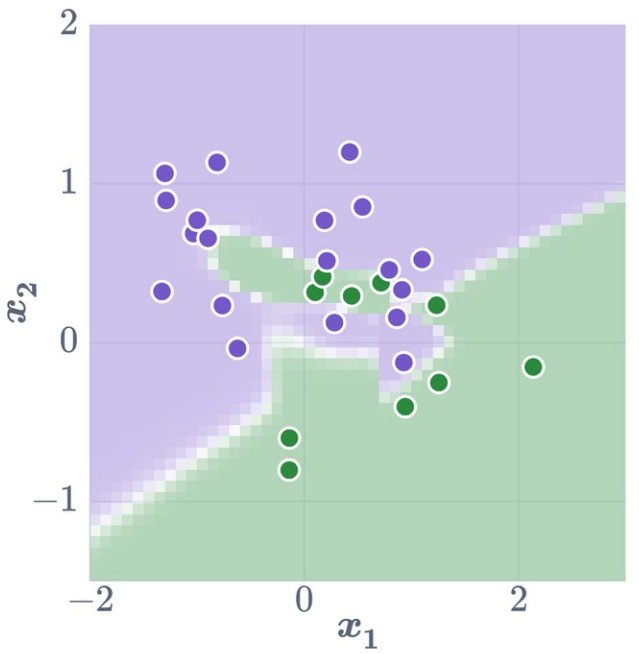
EPOCH	TRAINING ACCURACY	VALIDATION ACCURACY
3000	93%	77%

TRAINING IN PRACTICE

When to stop

- A maximum number of epochs; the weights stop changing; the loss is low enough.
- The training loss keeps going down.



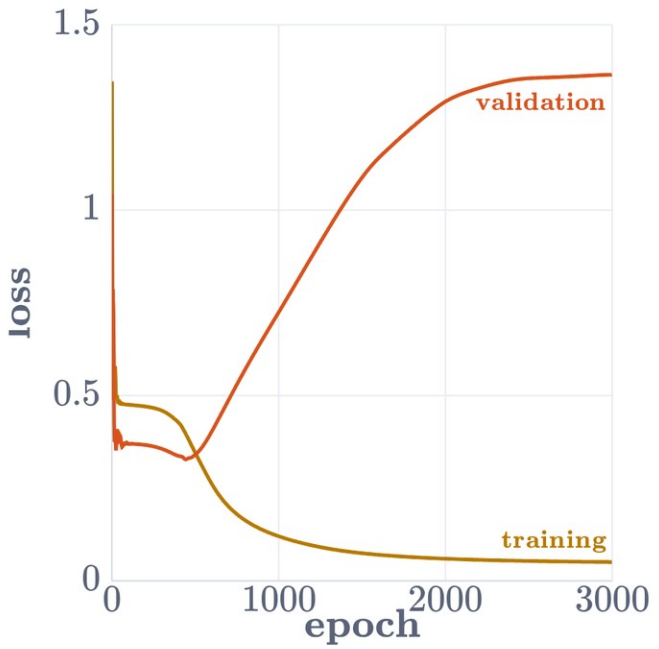


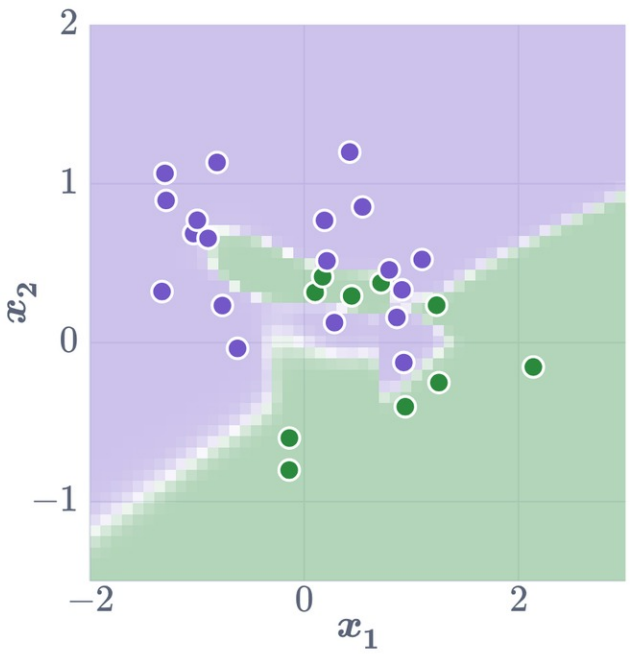
EPOCH	TRAINING ACCURACY	VALIDATION ACCURACY
3000	93%	77%

TRAINING IN PRACTICE

When to stop

- A maximum number of epochs; the weights stop changing; the loss is low enough.
- The training loss keeps going down.
- The validation loss goes down, then up: the network starts to overfit.



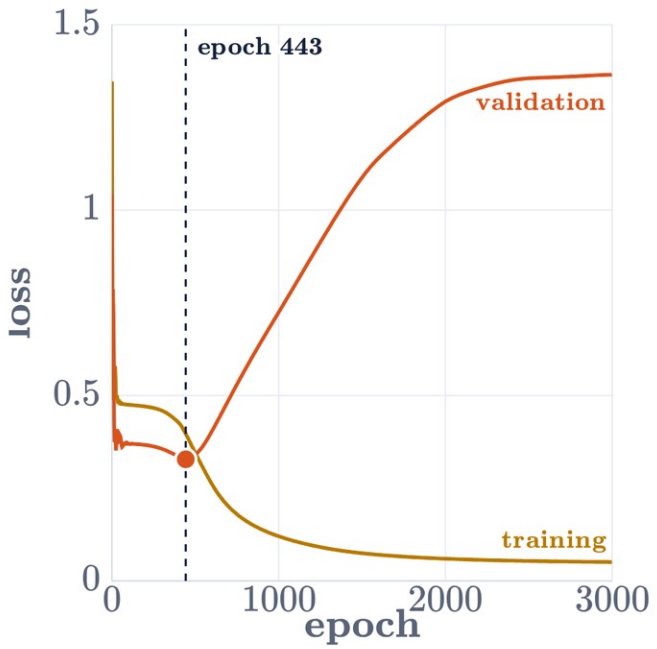


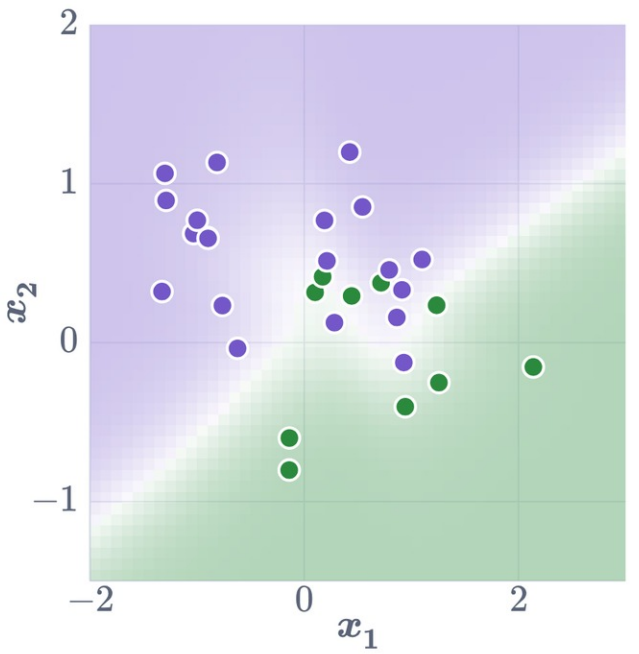
EPOCH	TRAINING ACCURACY	VALIDATION ACCURACY
3000	93%	77%

TRAINING IN PRACTICE

When to stop

- A maximum number of epochs; the weights stop changing; the loss is low enough.
- The training loss keeps going down.
- The validation loss goes down, then up: the network starts to overfit.
- **Early stopping**: keep the weights from the epoch with the lowest validation loss.



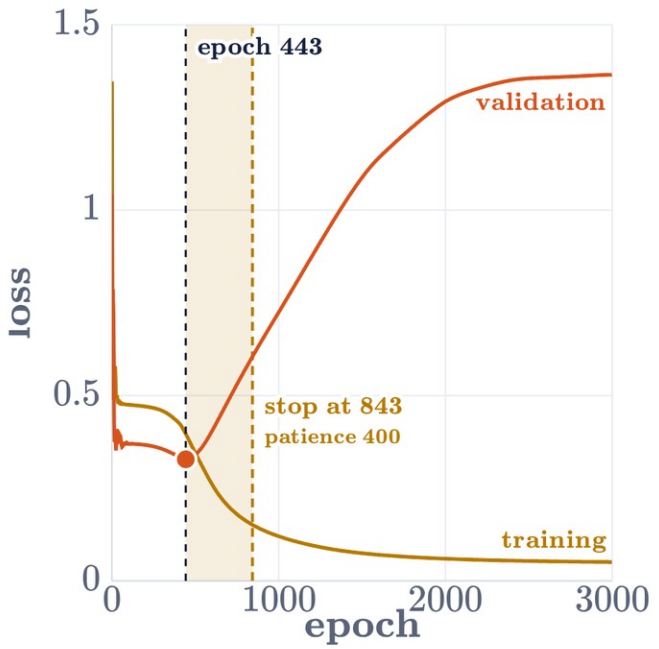


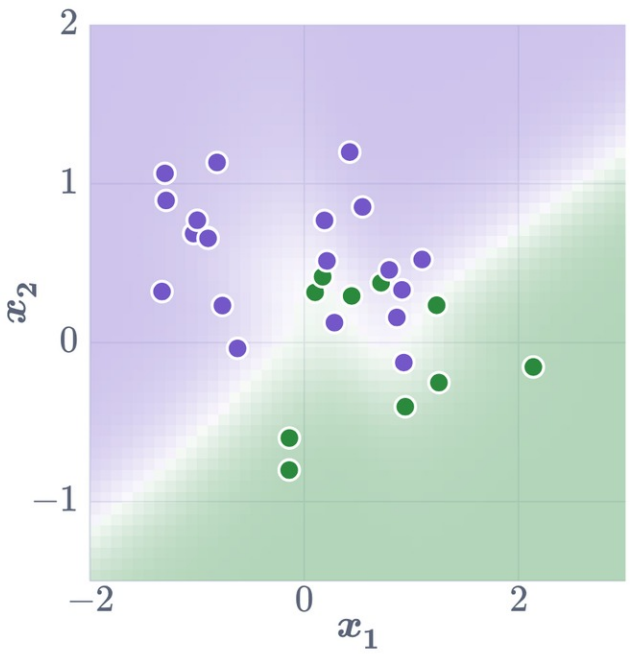
EPOCH	TRAINING ACCURACY	VALIDATION ACCURACY
443	80%	89%

TRAINING IN PRACTICE

When to stop

- A maximum number of epochs; the weights stop changing; the loss is low enough.
- The training loss keeps going down.
- The validation loss goes down, then up: the network starts to overfit.
- **Early stopping**: keep the weights from the epoch with the lowest validation loss.
- **Patience**: stop once the validation loss has not improved for a set number of epochs.





EPOCH	TRAINING ACCURACY	VALIDATION ACCURACY
443	80%	89%

SUMMARY

Neural networks: characteristics

Strengths

- Very flexible: one hidden layer with enough units can approximate any continuous function on a bounded region.
- They learn their own features, so less hand-made feature design.
- They improve with more data and computing power, and handle many outputs at once.
- Irrelevant inputs can be ignored: training can drive their weights to zero.

SUMMARY

Neural networks: characteristics

Strengths

- Very flexible: one hidden layer with enough units can approximate any continuous function on a bounded region.
- They learn their own features, so less hand-made feature design.
- They improve with more data and computing power, and handle many outputs at once.
- Irrelevant inputs can be ignored: training can drive their weights to zero.

Weaknesses

- Non-convex training: the answer depends on the start, with no guarantee of the best weights.
- Many choices to tune: layers, units, η, optimizer, epochs.
- They need a lot of data, overfit small data sets, and are hard to interpret.
- Training is time-consuming: many passes over the data, and many runs to tune the choices above.

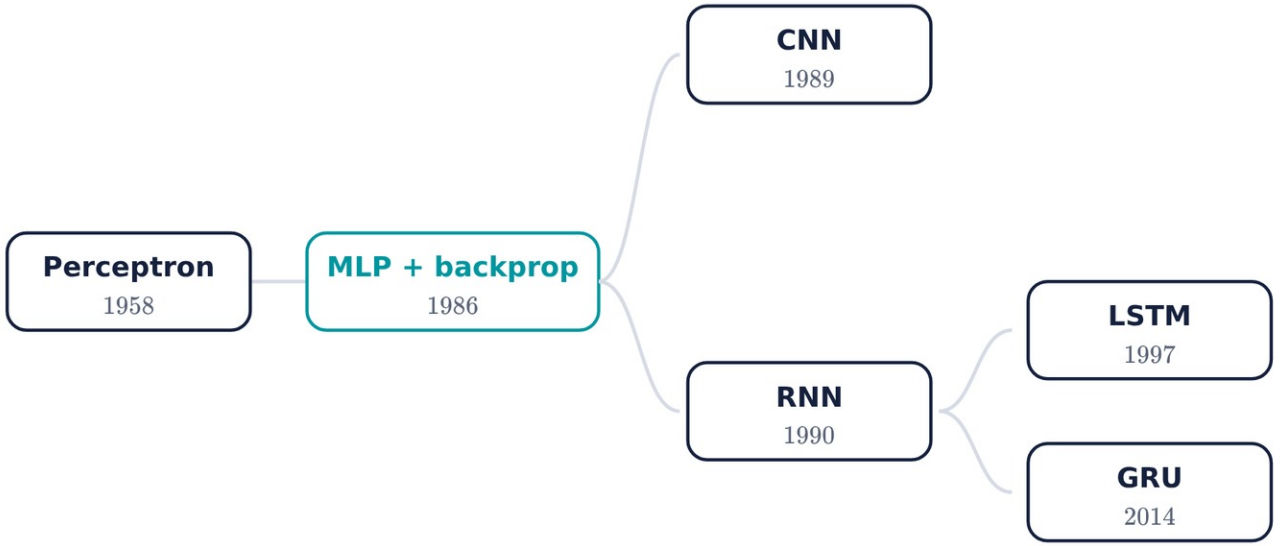
WHERE THIS GOES

The family tree



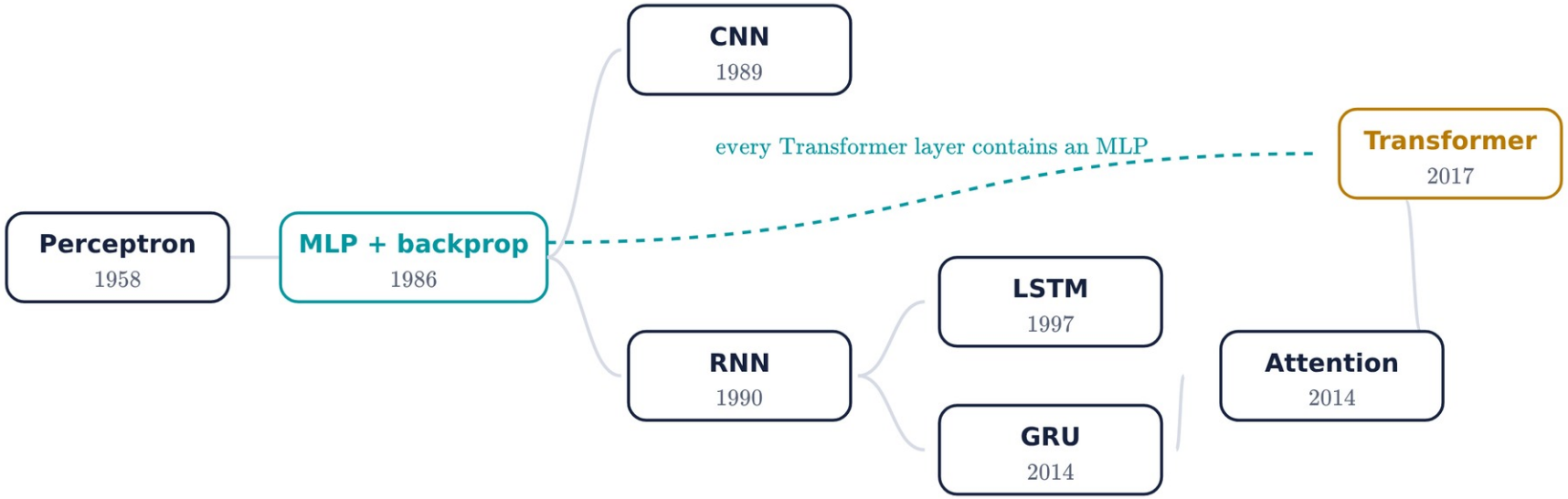
WHERE THIS GOES

The family tree



WHERE THIS GOES

The family tree



SUMMARY

Takeaways

Backpropagation
The chain rule, from the output back. $\delta^{(2)} = p - y$; each weight's gradient is δ at its end times the value at its start.

Optimizers
Learning-rate schedules, momentum, Adam.

Batches
Batch, mini-batch, stochastic. An epoch is one pass through the data.

Stopping
Watch the validation loss; stop early.

Thank you.

Questions?